Chapter 9

Scalability case studies

The scalability of a concurrent data structure is its ability to maintain or improve performance when increasing numbers of threads manipulate it concurrently. In this chapter we shall consider three case studies, which each shows that the same functionality may be implemented in different ways that exhibit widely different performance when the number of threads accessing them increases. The case studies are:

- Longadders: a single thread-safe `long` counter; see section 9.2.
- Concurrent hash maps: a thread-safe map from keys to values; see section 9.3.
- K-means clustering: forming clusters of points in the plane; see section 9.4.

Before the case studies, section 9.1 introduces virtual threads and how they differ from Java's classical threads and Java's futures and executor framework.

This chapter presents example code to illustrate implementation ideas and details. For brevity, we often leave out `private` and `public` modifiers as well as `final` modifiers on local variables and parameters, because these modifiers have no effect on the concurrent behaviour or performance of the code. But it is good software engineering practice to use them.

9.1 Platform threads, virtual threads and futures

Since Java version 21, there are two notions of thread: first, classical Java threads, also called platform threads; and secondly, virtual threads.

9.1.1 Platform threads versus virtual threads

Both platform threads and virtual threads are represented by class java.lang.Thread and therefore support the same operations, such as `join` and `start`, which is extremely convenient. The differences between them are these:

- A platform thread represents an underlying operating system thread, and is costly to create in time as well as space. One should create at most a few hundred of them.
- A virtual thread is much cheaper to create, both in time and space. Moreover, it handles blocking method calls better than futures in the executor framework do [3, sec. 6.1-6.2].

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

A new virtual thread is so cheap to create that one can make one just to execute a single input-output operation or database request. Indeed this is the intended application area, and the reason they have been designed to handle blocking operations well.

9.1.2 How virtual threads are executed

A virtual thread is executed by a platform thread, called a carrier thread. There is a small pool of such carrier threads, typically a fork-join pool with work stealing [18].

A virtual thread executes on its carrier thread until it performs a blocking operation, such as an input-output operation or database query. At that point it gets parked (as described in section 7.4) and detached from its carrier, and its execution stack gets copied to the heap (see section 8.2.3). When the blocking operation completes, a carrier thread, possibly different from the original one, resumes the execution of the virtual thread, restoring its execution stack from the heap.

This handling of blocking method calls sets virtual threads apart from tasks (or futures) in the executor framework. A task is classically executed by an underlying platform thread, and when the task performs a blocking operation, that underlying platform thread gets blocked also. Hence tasks are not practical for executing large numbers of blocking operations, but fine for CPU-intensive non-blocking operations. The properties and strengths of each mechanism are shown in figure 9.1.

Mechanism	Suitable for	Cost to create
Platform thread	All operations	High
Virtual thread	Blocking operations	Low
Task or future	Non-blocking operations	Low

Figure 9.1: Application areas and cost of creation for threads and tasks.

9.1.3 How to use virtual threads

Assume we want to download a few dozen webpages, given a list of URL strings. We can do it sequentially like this, where `getPage` is a blocking network operation:

```
static Map<String,String> getPagesSequential(String[] urls, int maxLines) {
  Map<String,String> result = new HashMap<String,String>();
  for (String url : urls) {
    try {
      result.put(url, getPage(url, maxLines));
    } catch (IOException exn) { ... }
  }
  return result;
}
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Since some webservers may be slow to reply, it may be much faster to send all the `getPage` requests in parallel using virtual threads, like this:

```
static Map<String,String> getPagesParallel(String[] urls, int maxLines) {
  Map<String,String> result = new ConcurrentHashMap<String,String>();
  List<Thread> threads = new ArrayList<Thread>();
  for (String url : urls) {
    String myUrl = url; // For capture in thread body
    threads.add(Thread.ofVirtual()
           .start(() ->
                  { try { result.put(myUrl, getPage(myUrl, maxLines)); }
                    catch (IOException exn) { ... }
                  }));
  }
  try {
    for (Thread t : threads)
      t.join();
  } catch (InterruptedException exn) { ... }
  return result;
}
```

If for some reason we would like to use platform threads instead of virtual threads, we would just have to replace `Thread.ofVirtual()` with `Thread.ofPlatform()` above. Both of these methods return a thread builder of class Thread.Builder; see section 9.1.4.

9.1.4 Thread builders and thread creation

To support the creation and handling of virtual threads, Java version 21 also introduced several new interfaces, including class Thread.Builder, and new static methods on class Thread:

- `Thread.ofPlatform()` returns a Thread.Builder for creating new platform threads.
- `Thread.ofVirtual()` returns a Thread.Builder for creating new virtual threads.

A Thread.Builder object `tb` supports several methods, including these:

- `tb.start(Runnable runn)` creates and starts a thread whose task is `runn`.
- `tb.unstarted(Runnable runn)` creates an unstarted thread whose task is `runn`.

Figure 9.2 compares some possible operations on classic platform threads and virtual threads. The operations are almost identical, except for two asymmetries: one can create a platform thread but not a virtual thread using a Thread constructor, and one can create and start a virtual thread but not a platform thread using a single Thread static method.

For symmetry and clarity it is probably good to always use a thread builder and its methods for thread creation, as in this idiom:

```
Thread th = Thread.ofPlatform().start(() -> { ... });
```

which makes it explicit what kind of thread one intends to create, and whether to immediately start it or not.

One advantage of thread builders is that one can write a method that takes as argument a thread builder `tb` and therefore is agnostic about the kind of threads being created. Method

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

	Platform threads	Virtual threads
Thread builder create	`tb = Thread.ofPlatform()`	`tb = Thread.ofVirtual()`
Thread create	`th = tb.unstarted(runn)`	`th = tb.unstarted(runn)`
Thread create	`th = new Thread(runn)`	
Thread create start	`th = tb.start(runn)`	`th = tb.start(runn)`
Thread create start		`th = Thread.startVirtualThread(runn)`
Get current thread	`Thread.currentThread()`	`Thread.currentThread()`
Thread join	`th.join()`	`th.join()`
Sleep current thread	`Thread.sleep(millis)`	`Thread.sleep(millis)`
Thread start	`th.start()`	`th.start()`
Get thread id as `long`	`th.threadId()`	`th.threadId()`
Yield to other threads	`Thread.yield()`	`Thread.yield()`

Figure 9.2: Some ways to create and manipulate thread builders, platform threads and virtual threads. Notation: `runn` is a Runnable, `tb` is a Thread.Builder, and `th` is a Thread.

`makeAndRun` below will create and start `threadCount` threads, each sleeping for 0.1 to 1.0 seconds, and then wait for all of them to finish:

```
static void makeAndRun(String kind, Thread.Builder tb, int threadCount) {
  Thread[] threads = new Thread[threadCount];
  for (int j=0; j<threadCount; j++)
    threads[j] = tb.start(() -> {
        try { Thread.sleep((int)(100 + 900 * Math.random())); }
        catch (InterruptedException exn) { }
      });
  try {
    for (int j=0; j<threadCount; j++)
      threads[j].join();
  } catch (InterruptedException exn) { }
  System.out.printf("%-8s count = %9d%n", kind, threadCount);
}
```

Now we can repeatedly call this method with a thread builder for platform threads, and double the number of threads `count` = 1, 2, 4, 8, 16, 32, ... until the program crashes, as a way to discover how many platform threads can be used simultaneously:

```
for (int count = 1; count < 100_000_000; count *= 2)
  makeAndRun("Platform", Thread.ofPlatform(), count);
```

As a practical observation we find (October 2026) that on a laptop with MacOS, 16 GB RAM and standard JVM settings, one can create and run 4,096 simultaneous platform threads. Attempting to create 8,192 platform threads will run out of operating systems threads and throw OutOfMemoryError, which is slightly misleading because the bottleneck is not memory. On the same machine, using `Thread.ofVirtual()` instead, one can create and run more than 4 million simultaneous virtual threads, but not 8 million of them. In other words, the Java runtime system can handle a thousand times more simultaneous virtual threads than platform threads. Creating, starting and joining a virtual thread seems to be around 50 times faster than creating, starting and joining a platform thread. A virtual thread seems to require less than 1 KB of memory.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

A practical mystery: Creating and starting but *not joining* a large number of virtual threads leads to much worse memory consumption than doing the same with platform threads. Apparently platform threads are garbage collected more effectively than virtual threads. The reason for this is currently unknown.

Finally, let us remark that while virtual threads are especially suitable for blocking operations, they can also be used for CPU-intensive non-blocking operations. However, they may incur more run-time overhead than tasks in the executor framework would, possibly due to the copying of the virtual thread's stack to heap, an operation not performed by tasks.

9.2 Long adder scalability case study

In this section we consider a very simple data structure: a 64-bit integer counter that can be safely incremented and read by multiple concurrent threads. A plausible use for such a data structure is to count events, such as requests to a web service, or elementary particles observed in a physics experiment. We present various implementations of this data structure, including Java's LongAdder class from package java.util.concurrent.atomic, we show that they have widely different scalability on shared-memory multicore machines, and we explain this in terms of the memory hierarchy and cache coherence protocol.

The data structures have this interface:

```
interface ILongAdder {
  void add(long delta);
  long longValue();
}
```

The expected behaviour is quiescent consistency: In the absence of concurrent calls to `add`, the result of `longValue` is the sum of the arguments `delta` passed in calls `add(delta)` by all threads until now. When there are ongoing calls to `add`, the result of `longValue` may not accurately reflect the completed calls to `add` neither at the initiation nor at the completion of the call to `longValue`. The long adder implementations we consider are:

LA1 SyncLongAdder: The baseline thread-safe implementation, a single 64-bit `long` field guarded by a single lock object.

LA2 SyncLongArrayAdder: An array of 64 `long` fields, each guarded by a separate lock; a thread with id `tid` updates the field with index `(tid % 64)`.

LA3 SyncLongArrayAdderPadded: Same as LA2 but with added memory padding.

LA4 WrappedAtomicLong: A single java.util.concurrent.atomic.AtomicLong field.

LA5 MyAtomicLong: An atomic long implemented with `compareAndSet` (CAS) in a loop.

LA6 ThreadLocalAtomicLong: An AtomicLong per thread, using Java's ThreadLocal<> class.

LA7 LongAdderArray: An AtomicLongArray indexed by `(tid % 64)`, similarly to LA2.

LA8 LongAdderArrayPadded: Same as LA7, but with added memory padding, as in LA3.

LA9 WrappedLongAdder: A wrapper around java.util.concurrent.atomic.LongAdder, which is carefully designed to be highly scalable.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

In the implementations (LA2, LA3, LA6, LA7, LA8, LA9) that use multiple `long` components, the `longValue` must be computed by summing the components. As noted above, we can only expect quiescent consistency of `longValue` in that case.

Some implementations — LA3, LA8 and LA9 — use added memory padding to reduce false sharing: the problem that different `long` fields may be on the same 64-byte cache line so that otherwise independent memory updates may incur cache coherence overhead. This is a case of false sharing; see section 8.7. More subtly, neighbouring *lock objects* could be on the same cache line, so taking two independent locks could also incur cache coherence overhead. As shown in chapter 7, taking a lock requires atomic updates to its internal fields.

Below, we first describe the various long adder implementations in more detail. Then we show experimental scalability results from different computers and interpret these results in the light of the hardware architecture.

9.2.1 LA1 SyncLongAdder implementation

The LA1 SyncLongAdder is the baseline thread-safe implementation, a single 64-bit `long` field guarded by a single lock object.

```
class SyncLongAdder implements ILongAdder {
  private long count = 0;
  private final Object lock = new Object();
  public void add(long delta) {
    synchronized (lock) {
      count += delta;
    }
  }
  public long longValue() {
    synchronized (lock) {
      return count;
    }
  }
}
```

The single private field `count` is a 64-bit long integer. All accesses to the field, whether reading or writing, happen after acquiring the intrinsic lock on the object bound to `lock`. This guarantees atomic increment in `add(delta)` and visibility of updates to other threads that call `longValue()`. Remember that `count += delta` is not itself atomic; it means `count = count + delta`, which would be prone to lost updates if not guarded by a lock.

There are two foreseeable performance problems with this long adder implementation: Due to the single lock, at most one thread can increment `count` at a time. But worse, every such thread must take the lock (which requires atomic access to a particular memory address in the lock object) and then the thread must write the `count` field (which invalidates a cache line of other cores running other threads), thus causing cache coherence overhead.

9.2.2 LA2 SyncLongArrayAdder implementation

The LA2 SyncLongArrayAdder is a refinement of LA1 SyncLongAdder using not a single `long` guarded by a single lock, but an array of 64 longs, each guarded by a separate lock.

```
class SyncLongArrayAdder implements ILongAdder {
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
  private final static int logS = 6, stripeCount = 1 << logS,
                           stripeMask = stripeCount - 1;
  private final long[] counts = new long[stripeCount];
  private final Object[] locks = new Object[stripeCount];

  public SyncLongArrayAdder() {
    for (int s=0; s<stripeCount; s++)
      locks[s] = new Object();
  }
  public void add(long delta) {
    int s = (int)Thread.currentThread().threadId() & stripeMask;
    synchronized (locks[s]) {
      counts[s] += delta;
    }
  }
  public long longValue() {
    long result = 0;
    for (int s=0; s<stripeCount; s++)
      synchronized (locks[s]) {
        result += counts[s];
      }
    return result;
  }
}
```

First note that when `logS` is 6, then `stripeCount` is $2^6 = 64$, and `stripeMask` is 63 which is $0b111111$ as a bitmask, so `(k & stripeMask)` is the same as `(k % 64)` for non-negative `k`.

Hence a thread with id `tid` takes the lock at `locks[tid % 64]` and updates the `long` at `counts[tid % 64]`.

The hope is that distinct threads, which have distinct thread ids `tid`, will mostly have distinct values of `(tid % 64)` and hence take distinct locks and update distinct fields.

Nevertheless, it is foreseeable that this does not achieve much speedup, because neighbouring `long` values such as `counts[0]` and `counts[1]` will likely be on the same cache line and cause false sharing. Indeed, a typical 64-byte cache will hold eight 8-byte `long` fields. The same is the case with the lock objects allocated by `locks[s] = new Object()`, although each Object might take up 16–24 bytes so there will be only 3 to 4 lock Objects on a cache line. Note that the `locks` array is only read, not written to, by the `add` and `longValue` methods. Therefore the cache lines holding the `locks` array can stay in the S(hared) state (see section 8.7) and the density of the `locks` array is not itself expected to ruin scalability.

Using `Thread.currentThread().threadId()` appears to be much faster than `hashCode()`, and in addition gives unique thread ids.

9.2.3 LA3 SyncLongArrayAdderPadded implementation

The LA3 SyncLongArrayAdderPadded works like LA2 but has added memory padding.

```
class SyncLongArrayAdderPadded implements ILongAdder {
  final static int logS = 6, stripeCount = 1 << logS,
                   stripeMask = stripeCount - 1, padding = 16;
  final long[] counts = new long[stripeCount * padding];
  final Object[] locks = new Object[stripeCount * padding];
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
  public SyncLongArrayAdderPadded() {
    for (int s=0; s<stripeCount * padding; s++)
      locks[s] = new Object();
  }

  public void add(long delta) {
    int s = (int)Thread.currentThread().threadId() & stripeMask,
        p = s * padding;
    synchronized (locks[p]) {
      counts[p] += delta;
    }
  }

  public long longValue() {
    long result = 0;
    for (int s=0; s<stripeCount; s++)
      synchronized (locks[s * padding]) {
        result += counts[s * padding];
      }
    return result;
  }
}
```

Class LA3 SyncLongArrayAdderPadded works in the same way as LA2 SyncLongArrayAdder, using an array of 64 longs and an array of 64 lock objects, but now 15 dummy longs have been added between every usable long in an attempt at avoiding false sharing between any two usable longs — those at `counts[0]`, `counts[16]`, `counts[32]`, and so on. Since a long takes 8 bytes, the padding between two neighbouring usable longs takes $15 * 8 = 120$ bytes, more than enough to push them to different cache lines. Likewise, 15 dummy lock objects are allocated between any two usable lock objects — those at `locks[0]`, `locks[16]`, `locks[32]`, and so on. The important effect is that 15 `new Object()` calls are performed between every two usable `new Object()` calls, hopefully achieving that the latter are not on the same cache line. In principle, the `locks` array does not need padding, since it is written to only once, in the constructor. However, it is probably important that the references from the `locks` array keep all the dummy lock objects alive. Otherwise the garbage collector might discard them, thereby possibly putting the usable lock objects back next to each other and reintroducing false sharing.

9.2.4 LA4 WrappedAtomicLong implementation

LA4 WrappedAtomicLong consists of a single java.util.concurrent.atomic.AtomicLong field.

```
class WrappedAtomicLong implements ILongAdder {
  private final AtomicLong atomicLong = new AtomicLong();

  public void add(long delta) {
    atomicLong.addAndGet(delta);
  }

  public long longValue() {
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
    return atomicLong.get();
  }
}
```

The documentation of Java's AtomicLong method `addAndGet(delta)` says "*Atomically adds the given value to the current value [...]*". It is a foreseeable performance problem that every thread that calls `addAndGet` must obtain atomic (exclusive) access to the same internal memory location in the AtomicLong object, thus causing cache coherence overhead.

In the Arm64 machine code generated by the Java JIT compiler (C2-compilation) for the `atomicLong.addAndGet(delta)` method call, this appears to be the crucial instruction:

```
ldaddal x2, xzr, [x10]
```

The Arm64 `ldaddal` instruction atomically loads a 64-bit doubleword from memory, here at the memory address in register `x10`; adds the value held in a register, here `x2` which contains `delta`, to it; and stores the result back to memory at address `x10`. The store to memory has release semantics; see section 8.5.

9.2.5 LA5 MyAtomicLong implementation

LA5 MyAtomicLong is implemented using AtomicLong's `compareAndSet` (CAS) in a loop:

```
class MyAtomicLong implements ILongAdder {
  private final AtomicLong atomicLong = new AtomicLong();

  public void add(long delta) {
    long oldValue, newValue;
    do {
      oldValue = atomicLong.get();
      newValue = oldValue + delta;
    } while (!atomicLong.compareAndSet(oldValue, newValue));
  }

  public long longValue() {
    return atomicLong.get();
  }
}
```

This is a way to implement an atomic add operation on a processor architecture that does not have an atomic increment instruction such as `ldaddal` above, but does have the universal compare-and-set (CAS) synchronization primitive, like most modern computers. The idea is to read the old value of a field using `get()`, add the old value and the `delta` to obtain `newValue`, and then atomically test whether the field still has the old value and if so set it to `newValue` and return true so the loop stops; otherwise update nothing and return false so the loop starts over.

The foreseeable performance problems here are twofold: First, like LA4 WrappedAtomicLong, every thread needs to write the same internal memory location (the one held in `x10`) inside the AtomicLong, thus causing cache coherence overhead. Secondly, if many concurrent threads are calling `add`, any given thread is likely to find that the old value has been overwritten, causing the `compareAndSet` to fail and the loop to start over. So in addition to the cache coherence overhead many cores may be busy executing instructions to no effect.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

If LA4 and LA5 have similar performance, then LA4 is probably implemented like LA5. If LA4 is much faster, its implementation likely uses an atomic-add machine code instruction such as the `ldaddal` above.

Here is part of the Arm64 machine code generated by the Java just-in-time compiler for the MyAtomicLong class `add(delta)` method. Various management code and null pointer checks have been left out. The argument `delta` is in register `x2` and the MyAtomicLong object reference is in register `x1`. The memory address of the actual long value to update inside the atomicLong is `([x1+12] << 3 + 0x10)`, computed by a three-instruction code sequence (`ldr`, `lsl`, `add`) e.g. in lines 52c4–52d0. Somewhat redundantly, there are four copies of this code.

The `do-while` loop has been partially unrolled. The Arm64 compare-and-set (CAS) instruction `casal` appears in lines 52f0 and 534c below. The code in lines 5300–5314 returns from the function when CAS is successful. The lines 5320–5364 form the retry loop when CAS was unsuccessful.

```
52c4: ldr   w11, [x1, #12]
52c8: lsl   x10, x11, #3
....
52d0: add   x10, x10, #0x10     // x10=address to update
52d4: ldar  x10, [x10]          // x10=oldValue=[x10]
52d8: ldr   w12, [x1, #12]
52dc: lsl   x11, x12, #3
52e0: add   x12, x2, x10        // x12=newValue=delta+oldValue
....
52e8: add   x11, x11, #0x10     // x11=address to update
52ec: mov   x8, x10             // x8=oldValue
52f0: casal x8, x12, [x11]      // if [x11]==oldValue set newValue
52f4: cmp   x8, x10
52f8: cset  x10, eq             // set x10 true if CAS successful
52fc: cbz   w10, 5318           // if CAS unsuccessful, try again
5300: ldp   x29, x30, [sp, #32] // else return
5304: add   sp, sp, #0x30
....
5314: ret
5318:
....
5320: ldr   w10, [x1, #12]
5324: lsl   x10, x10, #3
....:
532c: add   x10, x10, #0x10     // x10=address to update
5330: ldar  x12, [x10]          // x12=oldValue=[x10]
5334: ldr   w11, [x1, #12]
5338: lsl   x10, x11, #3
533c: add   x29, x12, x2        // x29=newValue=oldValue+delta
....
5344: add   x10, x10, #0x10     // x10=address to update
5348: mov   x8, x12             // x8=oldValue
534c: casal x8, x29, [x10]      // if [x10]==oldValue set newValue
5350: cmp   x8, x12
5354: cset  x10, eq             // set x10 true if CAS successful
....
5360: cbz   w10, 5320           // if CAS unsuccessful, try again
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
5364: b     5300                // else return
```

The `atomicLong.compareAndSet(oldValue, newValue)` Java method call is implemented by the instruction `casal x8, x12, [x11]` in line 52f0. Registers `x8` and `x10` both hold `oldValue`, register `x12` holds `newValue`, and register `x11` holds the memory address of the 64-bit long integer to update. The `casal` instruction atomically reads the value from memory address `[x11]`, compares it to `oldValue` in `x8`, and if equal, writes the `newValue` in `x12` to memory at address `[x11]`; otherwise leaves `[x11]` unchanged. It also puts the value read from `[x11]` into `x8`, so that if afterwards `x8` still equals `x10`, we know the CAS succeeded; otherwise it failed. The line 5340c instruction `casal x8, x29, [x10]` is another copy of the `compareAndSet` method call, just using different registers.

9.2.6 LA6 ThreadLocalAtomicLong implementation

LA6 ThreadLocalAtomicLong has an AtomicLong per thread, using the Java library ThreadLocal<> class.

```
class ThreadLocalAtomicLong implements ILongAdder {
  private final ConcurrentHashMap<AtomicLong,Boolean> allAtomicLongs =
    new ConcurrentHashMap<>();
  private final ThreadLocal<AtomicLong> atomicLongHolder =
    ThreadLocal.withInitial(() -> {
        AtomicLong al = new AtomicLong();
        allAtomicLongs.put(al, Boolean.TRUE);
        return al;
      });

  public void add(long delta) {
    atomicLongHolder.get().addAndGet(delta);
  }

  public long longValue() {
    return allAtomicLongs.keySet().stream().mapToLong(al -> al.get()).sum();
  }
}
```

The ThreadLocalAtomicLong implementation uses a separate AtomicLong for each thread that has called the `add` method. This should ensure even better memory separation between threads than in LA2 SyncLongArrayAdder and LA3 SyncLongArrayAdderPadded.

The ThreadLocal<> class works as follows. Static method `ThreadLocal.withInitial(supp)` calls the supplier function `supp` once for each thread that calls `add`. The `supp` function creates a new AtomicLong object, adds it to the `allAtomicLongs` set, and returns it. Thereby each thread gets its own separate AtomicLong object. A thread implicitly holds a reference to its own AtomicLong and can access it by calling `atomicLongHolder.get()`, as in method `add`.

The `allAtomicLongs` field is a set of all the AtomicLong objects belonging to the long adder. It is used by the `longValue` method to compute their sum. We use a hash map for this because Java does not have a ConcurrentHashSet.

The downside of the thread-local approach is that there will be as many AtomicLongs as there are distinct threads calling `add`. There could be thousands of such threads, which may cause high memory consumption and make `longValue` slow. This problem is made worse by

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

the `allAtomicLongs` set keeping AtomicLong objects live even after the threads they belonged to have died, but this is necessary for `longValue` to work. By contrast LA2 and LA3 allocate only 64 AtomicLongs regardless of the number of calling threads.

When the number of threads is modest (10–100), this implementation could be expected to perform very well.

9.2.7 LA7 LongAdderArray implementation

LA7 LongAdderArray consists of an AtomicLongArray, indexed by `(tid % 64)`, similarly to LA2.

```
class LongAdderArray implements ILongAdder {
  final static int logS = 6, stripeCount = 1 << logS,
                   stripeMask = stripeCount - 1;
  final AtomicLongArray counters = new AtomicLongArray(stripeCount);

  public void add(long delta) {
    counters.addAndGet((int)Thread.currentThread().threadId() & stripeMask,
                       delta);
  }

  public long longValue() {
    long result = 0;
    for (int s=0; s<stripeCount; s++)
      result += counters.get(s);
    return result;
  }
}
```

The LongAdderArray uses an AtomicLongArray with 64 atomic longs, using a logic very similar to LA2 SyncLongArrayAdder. The foreseeable performance problems are the same as for LA2, namely, that neighbouring atomic longs may be on the same cache line, causing false sharing and thereby cache coherence overhead.

The Arm64 machine code generated by the Java JIT compiler for LongAdderArray's `add` method involves a single `ldaddal` instruction and no loops, just like WrappedAtomicLong's `add` method:

```
2438: ldaddal x2, xzr, [x10]
```

The only difference is that the machine code (not shown) that computes the memory address to update (in register `x10`) is more complicated, as it involves array indexing based on the `threadId` and the `stripeMask`.

9.2.8 LA8 LongAdderArrayPadded implementation

LA8 LongAdderArrayPadded is the same as LA7, but with added memory padding, similarly to LA3.

```
class LongAdderArrayPadded implements ILongAdder {
  final static int logS = 6, stripeCount = 1 << logS,
                   stripeMask = stripeCount - 1, padding = 16;
  final AtomicLongArray counters
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
                    = new AtomicLongArray(stripeCount * padding);

  public void add(long delta) {
    counters.addAndGet(((int)Thread.currentThread().threadId() & stripeMask) *
        padding, delta);
  }

  public long longValue() {
    long result = 0;
    for (int s=0; s<stripeCount; s++)
      result += counters.get(s * padding);
    return result;
  }
}
```

The LA8 LongAdderArrayPadded uses padding between the atomic longs of LA7 LongAdderArray in an attempt to avoid false sharing and cache coherence overhead. Thus the relation between LA8 and LA7 is exactly the same as the relation between LA3 and LA2. However, whereas LA3 creates $16 * 64 = 1024$ lock objects, of which $15 * 64 = 960$ are never used, plus two array objects, LA8 creates only a single array object of length 1024. Moreover, whereas an aggressive garbage collector might reintroduce false sharing in LA3, this seems extremely implausible in LA8. There are no foreseen performance problems.

9.2.9 LA9 WrappedLongAdder implementation

LA9 WrappedLongAdder is a wrapper around a java.util.concurrent.atomic.LongAdder from the Java library.

```
class WrappedLongAdder implements ILongAdder {
  private final LongAdder longAdder = new LongAdder();
  public void add(long delta) {
    longAdder.add(delta);
  }
  public long longValue() {
    return longAdder.longValue();
  }
}
```

The LA9 WrappedLongAdder is a wrapper around the standard Java LongAdder, which is carefully designed and field-tested and can be expected to be highly scalable in most usage scenarios and on a range of computer architectures. The Java LongAdder implementation (by Doug Lea) is based on a rather advanced general Striped64 class, whose source code comments say "*this class maintains a lazily-initialized table of atomically updated variables, plus an extra base field. [...] Indexing uses masked per-thread hash codes. [...] Table entries are of class Cell; a variant of AtomicLong padded [...] to reduce cache contention.*" Thus atomic variables, striping by thread, CAS, and padding, combined with dynamic adaptation to the amount of contention; in total at least 200 lines of codes (apart from comments).

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

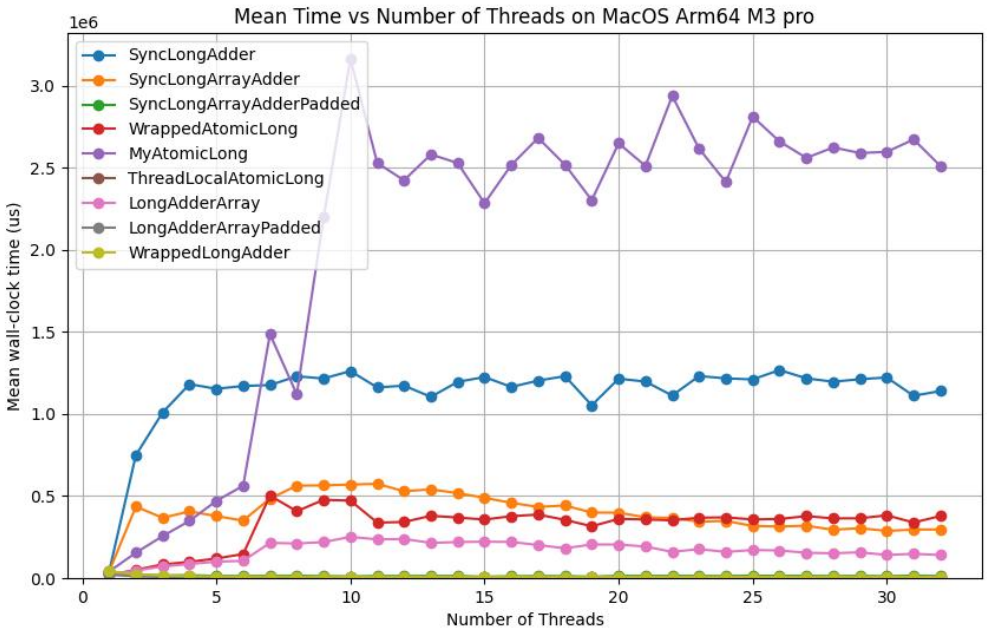


Figure 9.3: Time as function of number of workload threads, for all long adder implementations; lower is better. Apple M3pro.

9.2.10 Long adder scalability benchmarking

We shall measure the performance and scalability of all the nine long adders using the following setup. Each run creates a single long adder object (one of the implementations LA1–LA9) and runs `N` workload threads that concurrently execute this for-loop:

- for `i = 1, 2, ..., K` call `longAdder.add(i)`

We run this experiment for each thread count `N = 1, 2, ..., 32` and for each of the nine long adders, setting `K = 10,000,000 / N` so that each experiment performs (roughly) 10 million calls to `longAdder.add`. For each value of `N` we measure the wall-clock time for all workload threads to finish, averaged over 10 repetitions of the experiment. In this setup there are no calls to `longValue()` until all workload threads have terminated.

It is a sign of good scalability if the wall-clock time decreases with increasing thread count `N`, so long as `N` is less than the number of physical CPU cores. It is a sign of poor scalability if the wall-clock time stays constant or even increases with increasing thread count `N`.

9.2.11 Long adder measurements, Apple M3pro

We first run the long adder experiment on the Macbook laptop with an Arm64 M3pro processor whose structure is shown in figure 8.7.

From the figures 9.3 through 9.5 we can make four overall observations:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

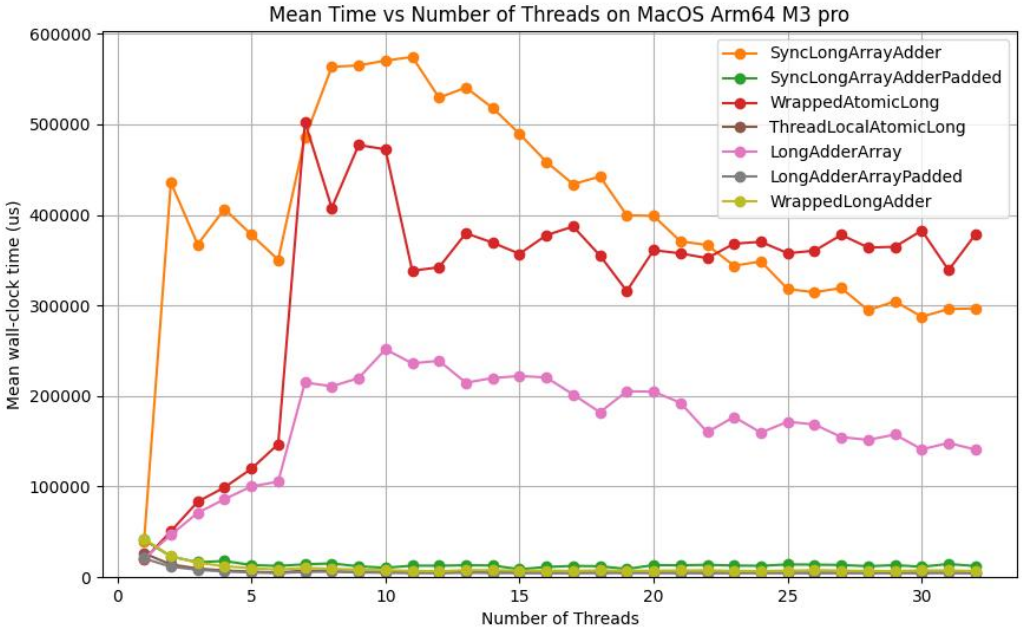


Figure 9.4: Time as function of number of workload threads, for long adder implementations except LA1 SyncLongAdder and LA5 MyAtomicLong; lower is better. Apple M3pro.

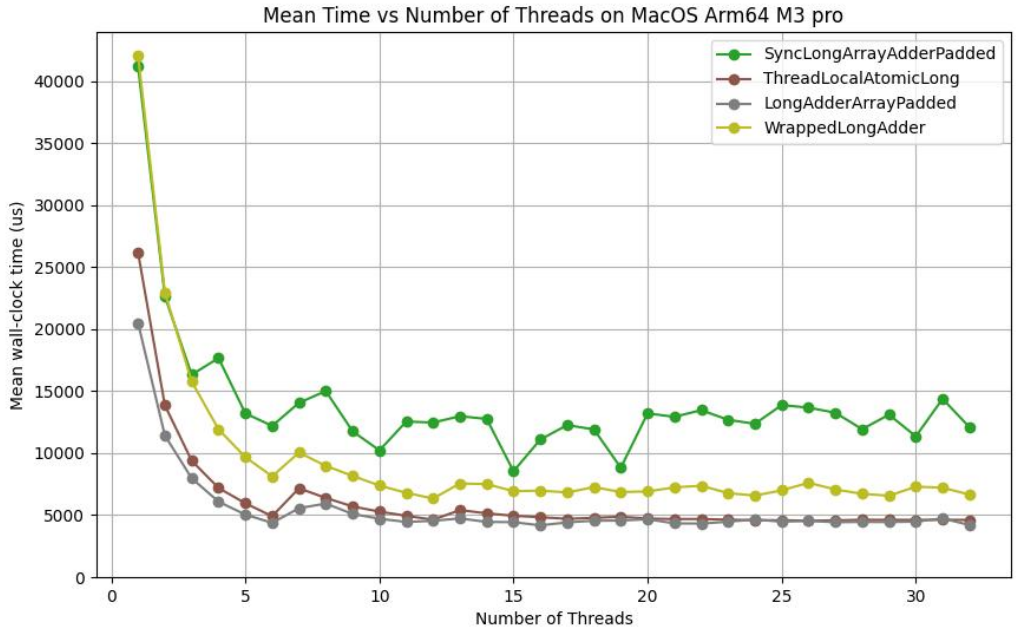


Figure 9.5: Time as function of number of workload threads, for the scalable long adder implementations LA3, LA6, LA8, LA9; lower is better. Apple M3pro.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

- First, figure 9.3 shows that for single-threaded execution (`N=1`), all long adder implementations LA1–LA9 are almost equally fast, taking 20–40 ms wall clock time, or 2–4 ns for each of the 10 million calls to `add`. This shows that the memory cache hierarchy works really well for sequential code execution. Presumably the single workload thread runs mostly on a single CPU core, and all updates can be made to that core's local caches, where all cache lines can remain in state M(odified), almost always; see section 8.7.

- Second, figure 9.3 also shows that LA5 MyAtomicLong does not scale at all for multiple threads. In particular, for `N=10` threads, its wall clock time is 3164 ms or 80 times slower than its single-threaded time, and this is despite using 10 cores instead of 1, so it actually performs roughly 800 times as much compute work to achieve the same result. Presumably there are two reasons for this: (a) repeated cache invalidation in each core, and (b) workload threads spend most of their time busy-waiting, undoing each others' work. This shows that `do-while` CAS-based concurrent algorithms can potentially be very inefficient. We will ignore LA5 MyAtomicLong from now on.

- Third, figure 9.4 shows that for high thread counts, the non-striped implementations (LA1 SyncLongArrayAdder and LA4 WrappedAtomicLong) and the striped but non-padded implementations (LA2 SyncLongArrayAdder and LA7 LongAdderArray) are vastly slower than the striped and padded ones (LA3 SyncLongArrayAdderPadded, LA6 ThreadLocalAtomicLong, LA8 LongAdderArrayPadded and LA9 WrappedLongAdder). This shows that avoiding cache coherence overhead is more important for multi-threaded scalability than whether the algorithm uses locks or low-level atomic operations such as atomic add in LA4 WrappedAtomicLong.

- Fourth, figure 9.4 shows that among the striped padded ones, LA8 LongAdderArray-Padded and LA7 ThreadLocalAtomicLong perform best and almost identically. Java's standard LA9 WrappedLongAdder is third-best and somewhat slower. The lock-based SyncLongArrayAdderPadded is the slowest of these four fastest ones, but still scales well with high thread counts. This shows that locks are not slow per se, and that for high thread (and core) counts, overhead from locking may be much smaller than from cache coherence.

Now let us consider the results for the various long adder implementations (except LA4).

- At single-threaded execution (`N=1`), the single-lock LA1 SyncLongAdder is as fast as most other long adders, but its performance degrades enormously when used from multiple threads. Already at `N=2` workload threads, SyncLongAdder takes 749 ms which is 19 times slower than its single-threaded time. This must be due to (a) contention for the single lock, and more importantly (b) when two or more cores try to take the same lock they will constantly invalidate each other's caches' copies of the memory location that holds the lock data. In particular, the cachelines holding the memory location representing the lock state will toggle between the M(odified) and I(nvalid) states. Another way of looking at this is that adding to a shared long integer is a very small amount of work for a CPU core, and distributing the 10 million add operations on multiple threads and multiple cores never pays off the overhead of taking a lock.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

- The striped multilock LA2 SyncLongArrayAdder is an improvement over the single-lock SyncLongAdder, but is not scalable: it is slower for multiple threads than for a single one. This may be attributed to the cache invalidation caused by the dense allocation of multiple long fields and multiple lock objects on the same cache lines (false sharing) and resultant cache invalidation. This is supported by comparing it with its padded version LA3 SyncLongArrayAdderPadded; see below.

- For 1 to 6 threads, LA4 WrappedAtomicLong is much faster than SyncLongAdder, then for 7–32 threads it runs the same speed, despite using only a single atomic add instruction (`ldaddal`) on Arm64, no locks, CAS or loops. Presumably, at high thread counts many cores attempt to perform the atomic add instruction, and the memory system can handle only a limited number of these at a time.

- It is especially instructive to compare the padded striped implementations LA3 SyncLongArrayAdderPadded and LA8 LongAdderArrayPadded with their non-padded striped versions LA2 SyncLongArrayAdder and LA7 LongAdderArray. The padded ones are vastly faster for high thread counts, that is, much more scalable. Since the only difference is that in the padded ones, the updatable locations are scattered in memory, the explanation for this must be that they avoid false sharing and thereby avoid repeated cache line invalidation. For a single thread (`N=1`), there is no significant difference between the wall-clock time of the padded and unpadded versions. Already at `N=2` threads, SyncLongArrayAdderPadded is 19 times faster than the unpadded version, and LongAdderArrayPadded is 4 times faster than the unpadded version. Astonishingly, at 11 threads SyncLongArrayAdderPadded is 46 times faster than the unpadded version, and at 12 threads LongAdderArrayPadded is 52 times faster than the unpadded version. This shows that vast inefficiencies can be introduced in parallel programming just by unfortunate use of shared mutable memory. Moreover, the fact that five of the long adder implementations are much slower than the lock-based LA3 SyncLongArrayAdderPadded shows that lock use is not the chief cause of inefficiency in multi-threaded programs: the CPU memory subsystem is the culprit.

- Overall, the LA8 LongAdderArrayPadded and LA6 ThreadLocalAtomicLong implementations are equally fast, and faster than all other implementations. The LA6 ThreadLocalAtomicLong guarantees that each thread has its own AtomicLong, so that mutable state is thread-local and thereby likely core-local. The downside is that if there are very many threads (rather than just 1–32) then it will allocate many AtomicLong objects and therefore use much memory. The 64-stripe LA8 LongAdderArrayPadded is a way to keep mutable state mostly thread-local and achieving very good performance without the downside of a ThreadLocalAtomicLong.

Finally, as shown in figure 8.7 the Apple M3 pro has 6 fast performance cores and 6 slower efficiency cores, and MacOS offers no way to control what threads run on which cores. The effect of this is visible in figure 9.5, where all four fast implementations have a local minimum wall-clock time at 6 threads (presumably running on the 6 performance cores) and a higher wall-clock time at 7 workload threads. Presumably the 7 threads run on 6 performance cores and 1 efficiency core, thus worsening performance. The LA4 WrappedAtomicLong has a particularly bad performance for 7 to 10 threads, corresponding to the use of the slower efficiency cores.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

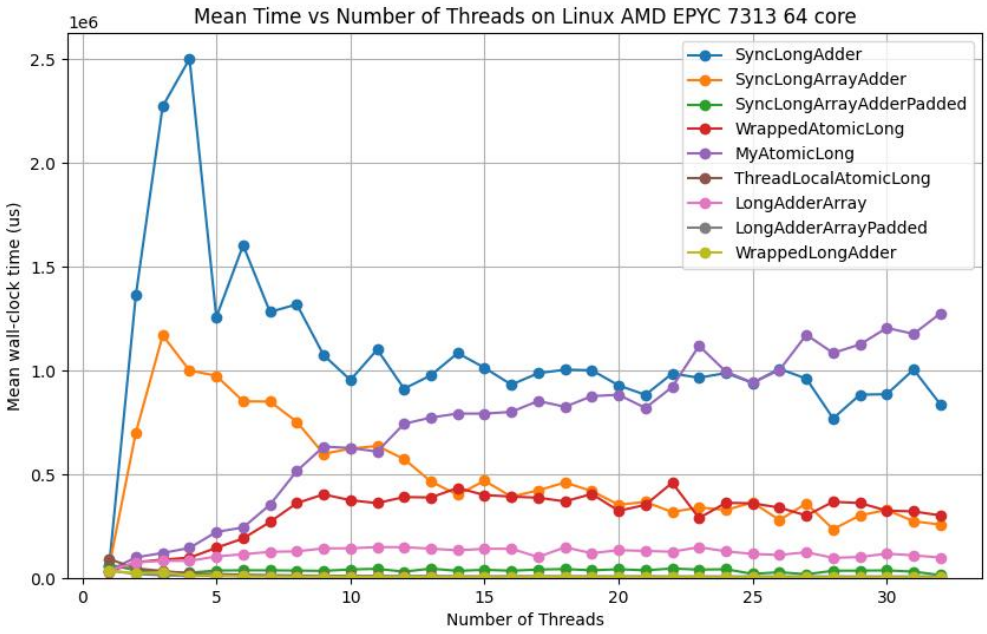


Figure 9.6: Time as function of number of workload threads, for all long adder implementations; lower is better. AMD EPYC 7313.

9.2.12 Long adder measurements, AMD EPYC 7313

The previous section analysed the performance of the long adder implementations on a highly integrated laptop CPU, the Apple M3 pro with 6 + 6 cores, which has all cores and all caches on the same chip. Here we consider a larger and more complex server CPU AMD EPYC 7313, whose structure is shown in figure 8.8. Each core supports AMD's "simultaneous multithreading" (see section 8.4.1), and can execute two threads at a time, but it is more realistic to consider this a 32-core machine than a 64-core machine. In any case, each core on the EPYC 7313 has its own L1 and L2 cache, and a block of 4 cores share an L3 cache. Cores in different blocks share only the memory bus and RAM; this may affect the performance of multi-threaded programs with much memory traffic.

Let us compare the experimental results for the Apple M3 pro (figures 9.3–9.5) with those of the AMD EPYC 7313 (figures 9.6 and 9.7)

With a single workload thread (`N=1`) all nine long adders are roughly equally fast, taking 30–65 ms to perform 10 million add operations, or roughly 3–6.5 ns per operation.

As before, MyAtomicLong becomes slower than all other implementations when there are many workload threads, presumably for the same reasons as before.

Disregarding MyAtomicLong, the remaining eight implementations fall in the same two groups as on the Apple M3 pro: those that become slower with increasing thread count, and those that become faster. As before, the slow group consists of LA1 SyncLongAdder, LA2 SyncLongArrayAdder, LA4 WrappedAtomicLong and LA7 LongAdderArray (in order from worst to best). The fast group consists of LA3 SyncLongArrayAdderPadded, LA8 LongAd-

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

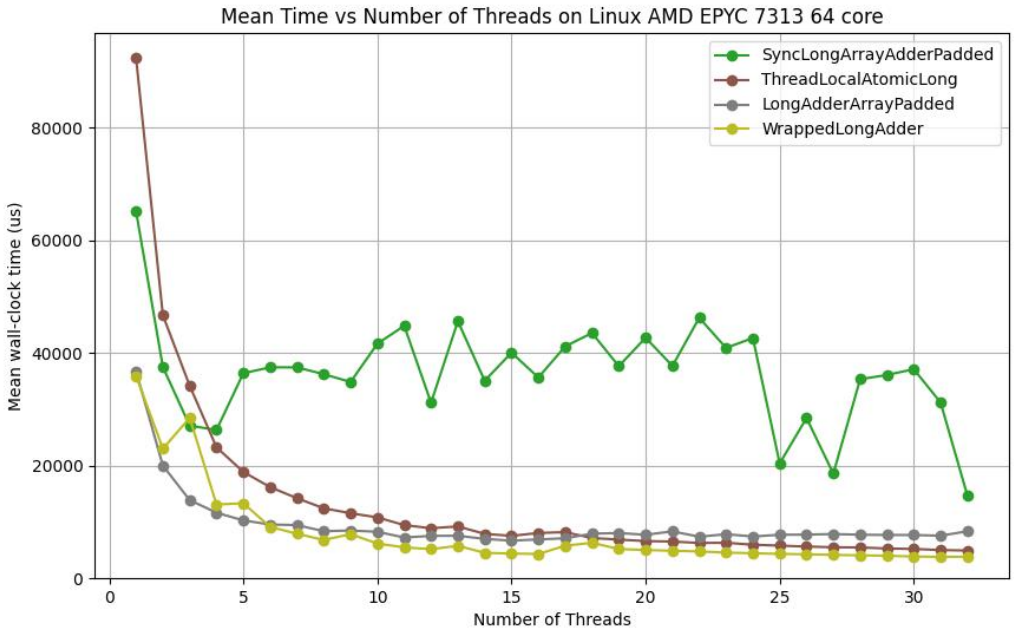


Figure 9.7: Time as function of number of workload threads, for the scalable long adder implementations LA3, LA6, LA8, LA9; lower is better. AMD EPYC 7313.

derArrayPadded, LA6 ThreadLocalAtomicLong, and LA9 WrappedLongAdder.

Hence on the server-class AMD EPYC 7313, as on the laptop-class Apple M3 pro, good scalability is achieved only when mutable memory is mostly thread-local and used in a cache-aware manner.

As before, the padded striped implementations LA3 SyncLongArrayAdderPadded and LA8 LongAdderArrayPadded are much faster than their non-padded striped versions LA2 SyncLongArrayAdder and LA7 LongAdderArray. For a single thread (`N=1`), there is no significant difference between the padded and unpadded versions. Already at `N=2` threads, SyncLongArrayAdderPadded is 19 times faster than the unpadded version, and LongAdderArrayPadded is 3.8 times faster than the unpadded version. At `N=16` threads, the padded striped versions are 11 times and 21 times faster than the unpadded ones, respectively.

As before, the LA3 SyncLongArrayAdderPadded implementation shows that lock-based solutions can be scalable and reasonably fast, provided mutable memory is mostly thread-local and cache-aware. However, the three atomic-add or compare-and-set-based cache-aware solutions are four times faster.

One notable difference is that on the AMD EPYC 7313 WrappedLongAdder, which is basically Java's LongAdder, consistently outperforms all the other long adder implementations. Presumably the Java LongAdder implementers focused on making LongAdder perform well on a wide range of hardware, including many-core server CPUs, whereas not much performance engineering has gone into our 15-line LongAdderArrayPadded.

The results are summarised in figure 9.8. It can be seen that both lock-based and atomic operation-based data structures can be scalable, but memory updates must be cache-local.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

				M3pro			AMD			
	Name	Sync	Local	1	32	Spd	1	32	Spd	Scble
LA1	SyncLongAdder	Lock	No	39	1140	0.03	55	838	0.07	No
LA2	SyncLongArrayAdder	Lock	No	39	297	0.13	55	258	0.21	No
LA3	SyncLongArrayAdderPadded	Lock	Yes	41	12	3.41	65	15	4.45	Yes
LA4	WrappedAtomicLong	Atom	No	19	378	0.05	31	302	0.10	No
LA5	MyAtomicLong	CAS	No	40	2512	0.02	36	1276	0.03	No
LA6	ThreadLocalAtomicLong	Atom	Yes	26	5	5.67	92	5	18.84	Yes
LA7	LongAdderArray	Atom	No	20	141	0.14	36	99	0.37	No
LA8	LongAdderArrayPadded	Atom	Yes	20	4	4.88	37	8	4.42	Yes
LA9	WrappedLongAdder	CAS	Yes	42	7	6.32	36	4	9.44	Yes

Figure 9.8: Overview of long adder implementations, their synchronization mechanism, cache-locality of updates, performance (ms per 10 million operations for 1 and 32 threads), speedup factor from 1 to 32 threads, and assessed scalability.

The cases LA1 SyncLongAdder, LA4 WrappedAtomicLong and LA5 MyAtomicLong show that lock-based, atomic and CAS-based operations all can have very bad scalability, when shared mutable memory is handled poorly.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

9.3 Concurrent hash map scalability case study

The long adder case study shows that even an extremely simple concurrent data structure — an integer counter — can be implemented in many different ways that have very different scalability on shared-memory multicore CPUs, and also shows that thread-locality of mutable memory updates is more important for performance than the choice of synchronization mechanism: lock, atomic add, or compare-and-set.

However, the case study could be critized for being too simplistic: the work carried out by each workload thread is a simple integer addition, though in a concurrent setting.

Here we therefore consider a more complex yet widely used data structure: concurrent hash maps, as implemented by e.g. Java's ConcurrentHashMap. Our concurrent hash map implementations must implement this interface:

```
public interface IMap<K,V> {
  boolean containsKey(K k);
  V get(K k);
  V put(K k, V v);
  V remove(K k);
  int size();
  void forEach(BiConsumer<K,V> consumer);
}
```

The methods are expected to work as for Java's ConcurrentHashMap. In particular, `size` is only quiescent consistent: the returned count can be relied on only if there are no concurrent modifications of the hash map. Also, if key `k` is already in the hash map, the value `v1` returned by `put(k, v2)` must be the value associated with `k` immediately before the update to `v2`, not some other value previously associated with `k`.

The concurrent hash map implementations we consider are:

HM1 SynchronizedMap: The baseline thread-safe implementation: a single lock guarding a sequential hash map implementation with an array of buckets, each bucket holding a mutable list of key-value pairs.

HM2 StripedMap: As HM1 but using lock striping, with 32 locks, each guarding every 32nd bucket.

HM3 StripedMapPadded: As HM2 but with memory padding in the locks array and between the lock objects.

HM4 StripedWriteMap: As HM2 but with immutable lists of immutable key-value pairs, so that lock taking is not needed for read access, only for updates (`put` and `remove`).

HM5 StripedWriteMapPadded: As HM4 but with memory padding in the locks array and between the lock objects.

HM6 StripedLevelWriteMap: A top-level array of stripes holds references to arrays of buckets, each bucket being a mutable list of key-value pairs; a kind of naturally padded HM2.

HM7 HashTrieMap: A hash trie using CAS-based operations, based on hash-array mapped tries [19].

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

HM8 WrapConcHashMap: a wrapper around a Java's highly scalable ConcurrentHashMap.

Below, we first describe the various hash map implementations in more detail. Then we show experimental scalability results from different computers and interpret these results in the light of the hardware architecture.

9.3.1 General hash map background

A hash map may be implemented as shown in figure 9.9 using a buckets array, where each entry refers to a linked list of key-value pairs. The hash map in the figure may represent a mapping from a student id such as 10406 to a student name such as "Joe".

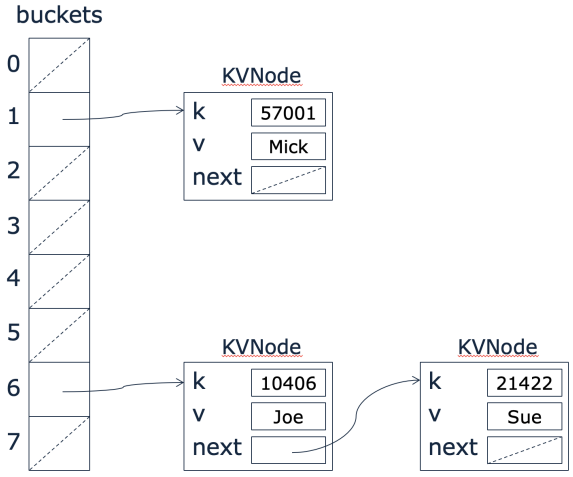


Figure 9.9: A hash map consisting of a buckets array, indexed by hash value modulo the aray length. A buckets array entry refers to a linked list of key-value pairs. Here bucket 1 contains only the pair (57001, "Mick"), bucket 6 contains the pairs (10406, "Joe") and (21422, "Sue"), and the other buckets are empty.

In most of our hash map implementations, a key-value pair is represented by an object of the KVPair class, with fields `k` and `v` and a `next` reference which is null at the end of the list:

```
static class KVPair<K,V> {
  private final K k;
  private V v;
  private KVPair<K,V> next;

  public KVPair(K k, V v, KVPair<K,V> next) { ... }

  public static <K,V> KVPair<K,V> search(KVPair<K,V> node, K k) {
    while (node != null && !k.equals(node.k))
      node = node.next;
    return node;
  }
}
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Method `search` searches a list starting at node for a key `k`, and returns the KVPair that holds `k`, if found, otherwise null. Note that the KVPair methods do not perform any synchronization, but leave that responsibility to the calling code — for instance, to take a suitable lock (or locks) before calling the `search` method.

9.3.2 HM1 SynchronizedMap implementation

HM1 SynchronizedMap is the baseline thread-safe implementation: a single lock guarding a sequential hash map implementation with an array of buckets, each bucket holding a mutable list of key-value pairs as in figure 9.9.

The state of a SynchronizedMap consists of the `buckets` array, which holds references of type KVPair, and the `cachedSize`, which is the number of key-value pairs in the buckets:

```
public class SynchronizedMap<K,V> implements IMap<K,V>  {
  private KVPair<K,V>[] buckets;
  private int cachedSize;
  ...
}
```

The synchronization policy is that all state (`buckets` and `cachedSize`) is guarded by the intrinsic lock of the SynchronizedMap. This is achieved by declaring all public SynchronizedMap methods synchronized, by declaring the state fields private, and by not letting any pointers to internal state escape, such as KVPairs references. This ensures that while one thread updates the hash map (by modifying `cachedSize` or `buckets` or data reachable from `buckets`) no other thread can modify or even read the hash map. It also ensures that data written by one thread will be visible to other threads subsequently accessing the hash map.

Putting the `synchronized` keyword in a method header is equivalent to wrapping the method body in this statement:

```
synchronized (this) { ... }
```

but we prefer to avoid cluttering the code.

The read access functions are easily implemented in terms of the `buckets` array and KVPair's `search` function. For instance, `containsKey(k)` first finds the hash of `k`, then finds the bucket number `buck = hash % buckets.length` where `k` belongs, then calls KVPair's `search` method on linked list `buckets[buck]` to search for `k`, and finally returns true if the search found `k` in the list:

```
public synchronized boolean containsKey(K k) {
  int hash = getHash(k), buck = hash % buckets.length;
  return KVPair.search(buckets[buck], k) != null;
}
```

The implementation of `get` is very similar, only in this case the value `v` from the KVPair is returned instead of true, and null is returned instead of false:

```
public synchronized V get(K k) {
  int hash = getHash(k), buck = hash % buckets.length;
  KVPair<K,V> node = KVPair.search(buckets[buck], k);
  if (node != null)
    return node.v;
  else
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
    return null;
  }
```

The `put(k, v)` method also calls KVPair's `search`, to either get the KVPair node that already contains `k` and then update that node's value field to `v`, or find that `k` is not in the hash map, and create a new KVPair and insert it in the appropriate bucket:

```
  public synchronized V put(K k, V v) {
    int hash = getHash(k), buck = hash % buckets.length;
    KVPair<K,V> node = KVPair.search(buckets[buck], k);
    if (node != null) {
      V old = node.v;
      node.v = v;
      return old;
    } else {
      buckets[buck] = new KVPair<K,V>(k, v, buckets[buck]);
      cachedSize++;
      if (cachedSize > buckets.length)
        reallocateBuckets();
      return null;
    }
  }
```

The `put` method also tests whether the hash map has become too full, and in that case calls `reallocateBuckets`, which doubles the length of the buckets array and redistributes the KVPairs over the new buckets array:

```
  public synchronized void reallocateBuckets() {
    KVPair<K,V>[] newBuckets = makeBuckets(2 * buckets.length);
    for (int buck=0; buck<buckets.length; buck++) {
      KVPair<K,V> node = buckets[buck];
      while (node != null) {
        int newBuck = getHash(node.k) % newBuckets.length;
        KVPair<K,V> next = node.next;
        node.next = newBuckets[newBuck];
        newBuckets[newBuck] = node;
        node = next;
      }
    }
    buckets = newBuckets;
  }
```

Although `reallocateBuckets` does not change the set of key-value pairs in the map, all other threads must be prevented from updating or reading the hash map in the middle of this drastic restructuring. This is one more reason for all methods to acquire the same intrinsic lock.

The `remove(k)` method should remove the target KVPair holding key `k`, if any. To do so, it finds the reference (from the buckets array or from another KVPair) that points to the target KVPair, and updates that reference to point to the successor of the target. In case of success, it must return the value `v` associated with `k` and also decrement the `cachedSize`.

```
  public synchronized V remove(K k) {
    int hash = getHash(k), buck = hash % buckets.length;
    KVPair<K,V> prev = buckets[buck];
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
    if (prev == null)
      return null;
    else if (k.equals(prev.k)) {        // Delete first KVPair
      V old = prev.v;
      cachedSize--;
      buckets[buck] = prev.next;
      return old;
    } else {                            // Search later KVPairs
      while (prev.next != null && !k.equals(prev.next.k))
        prev = prev.next;
      if (prev.next != null) {          // Delete KVPair prev.next
        V old = prev.next.v;
        cachedSize--;
        prev.next = prev.next.next;
        return old;
      } else
        return null;
    }
  }
```

The method `forEach(consumer)` calls `consumer(k,v)` for all key-value pairs in the hash map, one bucket's KVPairs at a time:

```
  public synchronized void forEach(BiConsumer<K,V> consumer) {
    for (int buck=0; buck<buckets.length; buck++) {
      KVPair<K,V> node = buckets[buck];
      while (node != null) {
        consumer.accept(node.k, node.v);
        node = node.next;
      }
    }
  }
```

The foreseeable performance problem with the HM1 SynchronizedMap is that a single lock is used to guard the entire hash map, so that only one thread can proceed to read or write or reallocate it at a time.

9.3.3 HM2 StripedMap implementation

HM2 StripedMap is much like HM1 SynchronizedMap but uses lock striping, with an array of 32 locks, each guarding every 32nd bucket in the buckets array.

This is expected to improve scalability because with some likelihood concurrent updates and reads will be on different stripes, will take different locks, and therefore can proceed at the same time.

```
  public class StripedMap<K,V> implements IMap<K,V> {
    private volatile KVPair<K,V>[] buckets;
    private final int lockCount;
    private final Object[] locks;
    private final int[] sizes;
    ...
  }
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

The synchronization policy is that `buckets[buck]` is guarded by `locks[buck%32]` for all bucket numbers `buck`, and that `sizes[strp]` is guarded by `locks[strp]` for all stripe numbers `strp` in 0–31. There is a `sizes[strp]` component for each stripe because a common size field could be a new bottleneck for scalability as shown by the long adder case study in section 9.2. The number of buckets must be a multiple of the number of stripes, so that a given key-value pair is always on the same stripe and always guarded by the same lock. The StripedMap uses the same KVPair class as before.

The StripedMap constructor allocates and initializes the `buckets`, `locks` and `sizes` arrays:

```
public StripedMap(int lockCount) {
  int bucketCount = lockCount; // Must be a multiple of lockCount
  this.lockCount = lockCount;
  this.buckets = makeBuckets(bucketCount);
  this.locks = new Object[lockCount];
  this.sizes = new int[lockCount];
  for (int strp=0; strp<lockCount; strp++)
    this.locks[strp] = new Object();
}
```

The `containsKey(k)` method illustrates how stripes are used for locking. Given the hash of `k`, the stripe `strp` of `k` is computed as `strp = hash % 32`, so the relevant lock is `locks[strp]` which is acquired before accessing the bucket `buck` which belongs to stripe `strp`:

```
public boolean containsKey(K k) {
  int hash = getHash(k), strp = hash % lockCount;
  synchronized (locks[strp]) {
    int buck = hash % buckets.length;
    return KVPair.search(buckets[buck], k) != null;
  }
}
```

Method `size()` needs to sum together all the stripes' size components:

```
public int size() {
  int result = 0;
  for (int strp=0; strp<lockCount; strp++)
    synchronized (locks[strp]) {
      result += sizes[strp];
    }
  return result;
}
```

Methods `get`, `put` and `remove` are as in SynchronizedMap, except they must use the same lock striping machinery as `containsKey`. Likewise, method `forEach` must use the same machinery as `size`, locking one stripe at a time. This means that `size` and `forEach` may or may note observe concurrent updates to the hash map, depending on which stripes they affect, but when there are no concurrent updates (quiescence) their behaviour is predictable.

By contrast, method `reallocateBuckets` must take all locks before redistributing key-value pairs on new buckets; otherwise there is a risk that an update is performed to an old bucket after its key-value pairs have been moved to the new buckets. The auxiliary method `lockAllAndThen(action)` will take all locks and then perform the action. The `action` is a lambda expression `() -> { ... }` that performs the actual reallocation from `oldBuckets` to `newBuckets`:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
public void reallocateBuckets(KVPair<K,V>[] oldBuckets) {
  lockAllAndThen(() -> {
      KVPair<K,V>[] bs = buckets;
      if (oldBuckets == bs) {
        KVPair<K,V>[] newBuckets = makeBuckets(2 * bs.length);
        for (int buck=0; buck<bs.length; buck++) {
          KVPair<K,V> node = bs[buck];
          while (node != null) {
            int newBuck = getHash(node.k) % newBuckets.length;
            KVPair<K,V> next = node.next;
            node.next = newBuckets[newBuck];
            newBuckets[newBuck] = node;
            node = next;
          }
        }
        buckets = newBuckets;
      }
    });
}
```

One subtlety that must be handled is that multiple threads may discover simultaneously that the hash map has become too full, and therefore call `reallocateBuckets` at almost the same time. Because of the lock taking, these calls obviously cannot run at the same time, but they can run one after the other. This might seem harmless, but if 10 threads each call `reallocateBuckets` to double the `buckets` array, it would become $2^{10} = 1024$ times larger instead of twice as large, and thereby use far too much memory. To prevent this from happening, `reallocateBuckets` first checks whether the StripedMap's `buckets` field still refers to the buckets array `oldBuckets` that existed when the call was made. If not, it does nothing. Hence only one of the simultaneous calls to `reallocateBuckets` will actually do anything.

In Java it seems necessary to use recursion to take the multiple intrinsic locks corresponding to all the stripes, so method `lockAllAndThen` is implemented like this:

```
private void lockAllAndThen(Runnable action) {
  lockAllAndThen(0, action);
}
private void lockAllAndThen(int strp, Runnable action) {
  if (strp >= lockCount)
    action.run();
  else
    synchronized (locks[strp]) {
      lockAllAndThen(strp + 1, action);
    }
}
```

Since the locks are always taken in the same order by `lockAllAndThen`, there is no risk of deadlock from multiple calls to `reallocateBuckets`.

The foreseeable performance problems with StripedMap are that the stripes of the buckets array are allocated next to each other, so that they may reside on the same cache line and cause false sharing between distinct stripes' bucket array entries. Moreover, neighbouring lock objects allocated by `locks[strp] = new Object()` may end up on the same cache lines.

This situation is the same as for the SyncLongArrayAdder class in section 9.2.2. The

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

obvious remedy, as in that case, is to add padding between the bucket elements and lock objects, which results in the StripedMapPadded that we consider next.

9.3.4 HM3 StripedMapPadded implementation

HM3 StripedMapPadded is like HM2 but with memory padding added to the `locks` array, between the lock objects, and to the `sizes` array (but not to the `buckets` array). The intention is to avoid false sharing between neighbouring lock objects (similar to LA3 SyncLongArrayAdderPadded), and between neighbouring size fields. The fields of a StripedMapPadded are those of a SynchronizedMap, plus a field defining a padding factor:

```
private final static int padding = 16;
```

This is used in the constructor to allocate 15 dummy lock objects between every two usable lock objects:

```
public StripedMapPadded(int lockCount) {
  int bucketCount = lockCount; // Must be a multiple of lockCount
  this.lockCount = lockCount;
  this.buckets = makeBuckets(bucketCount);
  this.locks = new Object[lockCount * padding];
  this.sizes = new int[lockCount * padding];
  for (int strp=0; strp<lockCount * padding; strp++)
    this.locks[strp] = new Object();
}
```

As in LA3 SyncLongArrayAdderPadded, having references from the `locks` array to the dummy lock objects probably helps prevent the garbage collector from discarding the dummy objects and moving the usable locks object next to each other, thereby effectively discarding the lock object padding.

The lock taking and size updates work precisely as before, but the stripe number used to index into the locks and sizes arrays must be multiplied by the padding factor 16. The `locks` array and `sizes` array accesses in method `put` are representative of this:

```
public V put(K k, V v) {
  int hash = getHash(k), strp = hash % lockCount;
  int afterSize = 0;
  V old = null;
  synchronized (locks[strp * padding]) {
    int buck = hash % buckets.length;
    KVPair<K,V> node = KVPair.search(buckets[buck], k);
    if (node != null) {
      old = node.v;
      node.v = v;
    } else {
      buckets[buck] = new KVPair<K,V>(k, v, buckets[buck]);
      afterSize = ++sizes[strp * padding];
    }
  }
  if (afterSize * lockCount > buckets.length)
    reallocateBuckets(buckets);
  return old;
}
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Everything apart from the `locks` and `sizes` allocation and their access remains unchanged. While the padding should remove false sharing from stripe lock taking and stripe size updates, there may still be false sharing between neighbouring bucket array entries (which are by definition on different stripes), and these entries are updated quite frequently when there is high proportion of `put` and `remove` operations on the hash map.

The padding factor of 16 may be unnecessarily large for the lock objects, where a factor of 4 or 6 might suffice, but is maybe on the small side for the `sizes` array of 32-bit integers. Making it an array of 64-bit longs might be better.

9.3.5 HM4 StripedWriteMap implementation

HM4 StripedWriteMap uses lock striping like HM2, but with immutable lists of immutable key-value pairs, so that lock taking is needed only for write access (`put` and `remove`); hence the name. No locks will be taken for read access (`containsKey`, `get`, `size`, `forEach`), which should improve scalability for read-heavy uses of the hash map.

In the new immutable key-value pair class ImKVPair, also fields `v` and `next` are declared final:

```
static class ImKVPair<K,V> {
  private final K k;
  private final V v;
  private final ImKVPair<K,V> next;
  ...
}
```

The immutable ImKVPair method `search(node, k, old)` works as before but no longer returns a reference to the ImKVPair node holding key `k` (if any). Instead it returns true if such a node exists and in that case passes back the value `v` associated with key `k` via the `old` parameter. This is basically an "out" parameter: an object whose only purpose is to hold a reference to `v` (or null):

```
static <K,V> boolean search(ImKVPair<K,V> node, K k, Holder<V> old) {
  while (node != null)
    if (k.equals(node.k)) {
      if (old != null)
        old.set(node.v);
      return true;
    } else
      node = node.next;
  return false;
}
```

Class Holder<V> is a very simple class with one field that can hold a reference to a value of type V, or null. It has methods `set` and `get`:

```
static class Holder<V> {
  private V value;
  public V get() { return value; }
  public void set(V value) { this.value = value; }
}
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

The immutable ImKVPair node class has a new method `delete(node, k, old)` which returns a version of the linked list without the node containing key `k`, if any, and passes the old value `v` back through the parameter `old`. The method will not update any existing ImKVPair, but may duplicate some of them to create the new list, and it returns the existing list (starting at `node`) if no changes were made:

```
static <K,V> ImKVPair<K,V> delete(ImKVPair<K,V> node, K k, Holder<V> old) {
  if (node == null)
    return null;
  else if (k.equals(node.k)) {
    old.set(node.v);
    return node.next;
  } else {
    ImKVPair<K,V> newNode = delete(node.next, k, old);
    if (newNode == node.next)
      return node;
    else
      return new ImKVPair<K,V>(node.k, node.v, newNode);
  }
}
```

The method is static, not an instance method, because it must be able to handle the case of an empty bucket, where `node` is null. The delete method is used by hash map methods `put(k,v)` and `remove(k)` to remove the old value associated with key `k`, if any.

The HM4 StripedWriteMap itself has these fields:

```
public class StripedWriteMap<K,V> implements IMap<K,V> {
  private volatile ImKVPair<K,V>[] buckets;
  private final int lockCount;
  private final Object[] locks;
  private final AtomicIntegerArray sizes;
  ...
}
```

The synchronization policy is as follows: writing to `buckets[buck]` is guarded by taking lock `locks[buck % lockCount]`. Read operations do not take locks at all, as they access only immutable values. The visibility of writes to reads is ensured by writes always writing to the stripe's size component after operations even if they do not change the size, and reads always reading from the stripe's size component before they access any data that could have been written. It follows from Java's visibility rules for AtomicIntegerArray that all write operations preceding the write to sizes will be visible to all read operations following the read from sizes. This is demonstrated by method `get` reading sizes before accessing array `bs[buck]`:

```
public boolean containsKey(K k) {
  ImKVPair<K,V>[] bs = buckets;
  int hash = getHash(k), strp = hash % lockCount, buck = hash % bs.length;
  return sizes.get(strp) != 0 && ImKVPair.search(bs[buck], k, null);
}
```

Since the `buckets` field is declared volatile, reading it `(bs = buckets)` is guaranteed to get the latest value written by the constructor or by `reallocateBuckets`, even if accessed before reading the `sizes` field.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Method `put(k,v)` first takes the relevant stripe's lock, then calls `delete(k)` on the relevant bucket's node list to remove any existing entry for `k`, then creates an ImKVPair to represent `(k,v)` and adds it at the head of the list. Subsequently it writes to `sizes` regardless of whether the stripe's size was incremented or not; this ensures visibility to subsequent read operations even though they do not take any locks:

```
public V put(K k, V v) {
  int hash = getHash(k), strp = hash % lockCount;
  Holder<V> old = new Holder<V>();
  ImKVPair<K,V>[] bs;
  int afterSize;
  synchronized (locks[strp]) {
    bs = buckets;
    int buck = hash % bs.length;
    ImKVPair<K,V> node = bs[buck],
      newNode = ImKVPair.delete(node, k, old);
    bs[buck] = new ImKVPair<K,V>(k, v, newNode);
    afterSize = sizes.addAndGet(strp, newNode == node ? 1 : 0);
  }
  if (afterSize * lockCount > bs.length)
    reallocateBuckets(bs);
  return old.get();
}
```

Avoiding all lock taking for read access (`containsKey`, `get`, `size`, `forEach`) may radically increase scalability because any number of read operations can proceed concurrently, regardless of the number of stripes. This may be especially important for the `forEach` method, which can have an arbitrarily long execution time. The downside is that updating the node lists may create a large number of ImKVPair clones, so immutability is likely to pay off performance-wise only when there is a fairly low proportion of update operations.

Other foreseeable performance problems include possible false sharing between the densely allocated stripe lock objects in the locks array, as well as false sharing between the stripe size components in the sizes AtomicIntegerArray, exactly as observed for LA7 LongAdderArray in section 9.2.11. As a proposed solution to these problems, we therefore next add padding between the lock objects and the size components, to obtain the HM5 StripedWriteMapPadded.

9.3.6 HM5 StripedWriteMapPadded implementation

HM5 StripedWriteMapPadded is like HM4 but with memory padding added to the `locks` array and between the lock objects. The same immutable ImKVPair key-value nodes are used as in HM4. The difference from the unpadded HM4 StripedWriteMap should by now be predictable: add a final field stating the padding factor, here 16, so 15 dummy fields and lock objects are allocated between any two usable fields and objects:

```
public class StripedWriteMapPadded<K,V> implements IMap<K,V> {
  private volatile ImKVPair<K,V>[] buckets;
  private final int lockCount;
  private final Object[] locks;
  private final AtomicIntegerArray sizes;
  private final static int padding = 16;
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
  public StripedWriteMapPadded(int lockCount) {
    int bucketCount = lockCount; // Must be a multiple of lockCount
    this.lockCount = lockCount;
    this.buckets = makeBuckets(bucketCount);
    this.locks = new Object[lockCount * padding];
    this.sizes = new AtomicIntegerArray(lockCount * padding);
    for (int strp=0; strp<lockCount * padding; strp++)
      this.locks[strp] = new Object(); // includes padding objects
  }
  ...
}
```

Note that there is no padding between the entries of the buckets array. The methods are similarly changed in a predictable way relative to StripedWriteMap. For instance, in `containsKey`, `get`, `size` and `forEach`, the access to the `sizes` array is padded:

```
public int size() {
  int result = 0;
  for (int strp=0; strp<lockCount; strp++)
    result += sizes.get(strp * padding);
  return result;
}
```

In the `put` and `remove` methods, also the lock taking is padded, but nothing else changes:

```
public V put(K k, V v) {
  int hash = getHash(k), strp = hash % lockCount;
  Holder<V> old = new Holder<V>();
  ImKVPair<K,V>[] bs;
  int afterSize;
  synchronized (locks[strp * padding]) {
    bs = buckets;
    int buck = hash % bs.length;
    ImKVPair<K,V> node = bs[buck],
      newNode = ImKVPair.delete(node, k, old);
    bs[buck] = new ImKVPair<K,V>(k, v, newNode);
    afterSize = sizes.addAndGet(strp * padding,
                                newNode == node ? 1 : 0);
  }
  if (afterSize * lockCount > bs.length)
    reallocateBuckets(bs);
  return old.get();
}
```

The foreseeable performance problems include the extra allocation costs for write-heavy uses of the hash map, as in HM4 StripedWriteMap, and also the possible false sharing between neighbouring buckets array entries as in HM3 StripedMapPadded and HM4 StripedWriteMap.

The next hash map implementation attempts to avoid the latter while retaining striping and immutability.

9.3.7 HM6 StripedLevelWriteMap implementation

HM6 StripedLevelWriteMap is a hash map where a top-level array holds references to Stripe objects, and a Stripe object holds an entire stripe's bucket array and its AtomicInteger size

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

field. Each bucket in turn is an immutable list of immutable key-value pairs, using ImKVPair as in HM4 and HM5. Thus instead of having a single array of buckets which are on alternating stripes, we have multiple bucket arrays, one per stripe. Thereby a stripe's bucket entries are all next to each other, and therefore rarely next to other stripes' bucket entries. As a consequence, the buckets arrays should require no padding, and the same should hold for the stripes' size fields.

The StripedLevelWriteMap and its nested Stripe class are declared as follows:

```
public class StripedLevelWriteMap<K,V> implements IMap<K,V> {
  private final Stripe<K,V>[] stripes;
  private final int lockCount;

  private static class Stripe<K,V> {
    private volatile ImKVPair<K,V>[] buckets;
    private final AtomicInteger size;
    ...
  }
}
```

The synchronization policy is that all writes to the data of a stripe, that is, to the elements of the `stripe.buckets` array, are guarded by the Stripe object's intrinsic lock. Read operations can proceed without taking locks because all data except the `stripe.buckets` array are immutable. The visibility of writes to reads is ensured by writes ending by writing to the stripe's `size` component (even if size does not change) and reads starting by reading from that same component. Since it is an AtomicInteger, it has the same visibility effects as a write to a volatile field followed by a read from it.

The `containsKey(k)` method starts by splitting the hash value `hash` into its least significant bits (the stripe number `strp = hash % 32`) and the remaining more significant bits `(rest = hash / 32)`, looks up the `stripe = stripes[strp]`, gets the stripe's buckets array `bs = stripe.buckets`, and finally indexes into `bs` using `rest` modulo the number of buckets:

```
public boolean containsKey(K k) {
  int hash = getHash(k), strp = hash % lockCount, rest = hash / lockCount;
  Stripe<K,V> stripe = stripes[strp];
  ImKVPair<K,V>[] bs = stripe.buckets;
  return stripe.size.get() != 0
         && ImKVPair.search(bs[rest % bs.length], k, null);
}
```

The `stripes` field is final and therefore visible to the thread calling `containsKey`. The `stripe.buckets` field is volatile, and updates to it (from reallocating the buckets of a stripe) therefore visible. The indexing into the `stripe.bucket` array is preceded by a read of the stripe's `size` field of type AtomicInteger which is written to after any update to that array, making these writes visible to the indexing. The same holds for all other read accesses to the StripedLevelWriteMap, in `get`, `size`, and `forEach`.

An updating operation, such as `put(k,v)`, follows the same initial pattern to determine the stripe, and then takes the stripe's intrinsic lock before any manipulation of the stripe's buckets:

```
public V put(K k, V v) {
  int hash = getHash(k), strp = hash % lockCount, rest = hash / lockCount;
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
    Stripe<K,V> stripe = stripes[strp];
    synchronized (stripe) {
      ImKVPair<K,V>[] bs = stripe.buckets;
      Holder<V> old = new Holder<V>();
      int buck = rest % bs.length;
      ImKVPair<K,V> node = bs[buck],
                    newNode = ImKVPair.delete(node, k, old);
      bs[buck] = new ImKVPair<K,V>(k, v, newNode);
      int afterSize = stripe.size.addAndGet(newNode == node ? 1 : 0);
      if (afterSize > bs.length)
        reallocateBucket(stripe);
      return old.get();
    }
  }
```

After creating an item list `newNode` without key `k`, it puts the key-value pair `(k,v)` in front of that list, updates the bucket array entry, and writes to the stripe's `size` field for visibility. It finally checks whether this stripe's bucket array has become too full, and if so reallocates (only) that. Method `remove` is similar.

The code for `reallocateBucket` is simpler than before. One reason is that the thread executing `put` already holds the stripe's lock and does not need to take any other locks. Another reason is that no other thread can try to reallocate this stripe's buckets at the same time, since no other thread can add to the stripe between this thread discovering the need for reallocation and this thread performing the reallocation.

```
  public void reallocateBucket(Stripe<K,V> stripe) {
    ImKVPair<K,V>[] bs = stripe.buckets,
      newBuckets = makeBuckets(2 * bs.length);
    for (int buck=0; buck<bs.length; buck++) {
      ImKVPair<K,V> node = bs[buck];
      while (node != null) {
        int newBuck = (getHash(node.k) / lockCount) % newBuckets.length;
        ImKVPair<K,V> tail = newBuckets[newBuck];
        newBuckets[newBuck] = node.next==tail ? node
                            : new ImKVPair<K,V>(node.k, node.v, tail);
        node = node.next;
      }
      stripe.buckets = newBuckets;
    }
  }
```

As for HM4 StripedWriteMap and HM5 StripedWriteMapPadded, since read operations do not take locks, any number of them can proceed concurrently, and at the same time as write operations, which should improve scalability. A further advantage of HM6 StripedLevelWriteMap is that buckets can be reallocated stripewise, thus causing shorter pauses in write access to the hash map (because not all locks have to be taken before, and held during, reallocation) and possibly also less memory usage.

One foreseeable performance problem of the design may be that each bucket access requires two array accesses: first look up the relevant stripe object, then look up the relevant bucket in that stripe's bucket array. But since the top-level stripes array is allocated only once and its elements are rarely updated (namely, only at bucket reallocations) and frequently used (on

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

each read or write to the hash map) it can remain in the S(hared) state in most cores' caches most of the time.

9.3.8 HM7 HashTrieMap implementation

HM7 HashTrieMap is a hash trie using CAS-based operations, based on hash-array mapped tries [19].

This hash map implementation is very different from the preceding ones. Instead of arrays of buckets of lists of key-value pairs, it uses a *hash trie*, a tree structure in which leaves hold the key-value pairs and each internal node holds an array of the next lower level subtrees. To locate a key in the hash trie, its hash code is divided into 6-bit fragments and each of these fragments is used to index into an internal node's array (64-way because $2^6 = 64$) until one reaches a key-value pair at a leaf. Also, instead of locks, the hash trie uses compare-and-set operations to update nodes.

The hash trie design is due to Prokopec, Bagwell and Odersky [19], and is one of two collections of concurrent hash maps in Scala's library; the other is Java's ConcurrentHashMap. The implementation considered here is by Florent Le Gall [11]. The implementation is considerably more complex and subtle than those considered so far. As a crude measure, our HM6 StripedLevelWriteMap is around 240 lines of code, and HM7 HashTrieMap is around 1070 lines of code.

9.3.9 HM8 WrapConcHashMap implementation

HM8 WrapConcHashMap is a thin wrapper around Java's standard ConcurrentHashMap.

The Java ConcurrentHashMap implementation is very advanced: it uses both locks and compare-and-set operations; its read operations do not take locks, like the "WriteMap" implementations HM4, HM5 and HM6 above; it is careful about avoiding cache coherence overhead; when hash collisions lead to too-large buckets, it attempts to build a binary tree for faster key-value pair access; and much more.

The implementation comprises 6400 lines of code, including ample comments [17]. The implementation evolved quite substantially from Java 6 to Java 7 to Java 8, presumably based on experience in industrial use, and as a result it is highly scalable and robust. It was written by Doug Lea and other authors of the Goetz textbook [3]. If you are really interested in scalable data structures, it is worth reading the source comments.

9.3.10 Hash map scalability benchmarking

We shall measure the performance and scalability of all eight hash maps using the following setup. Each benchmark run uses `N` workload threads that concurrently execute this for-loop:

```
for i = 1, 2, ..., K do
  key = random()
  if !map.containsKey(key)
    with probability 60% do map.put(key, key as string)
  else
    with probability 10% do map.remove(key)
```

We run this experiment for each thread count `N = 1, 2, ..., 32` and for each of the eight hash maps, setting `K = 5,000,000 / N` so that each experiment performs (roughly) 5 million

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

calls to `containsKey` and smaller numbers of calls to `put` and `remove`. For each value of `N` we measure the wall-clock time for all workload threads to finish, averaged over the 10 repetitions of the experiment.

The 60 % probability of adding a key if not in the hash map and the 10 % probability of deleting it if it is the hash map together ensure a rather high update frequency, probably more than many hash maps will experience in reality. This is chosen to show that even the implementations that use immutable data can perform reasonably well in such a scenario.

9.3.11 Hash map measurements, Apple M3pro

This section presents and analyses the performance of the eight hash map implementations the Macbook laptop with an Arm64 M3pro processor whose structure is shown in figure 8.7.

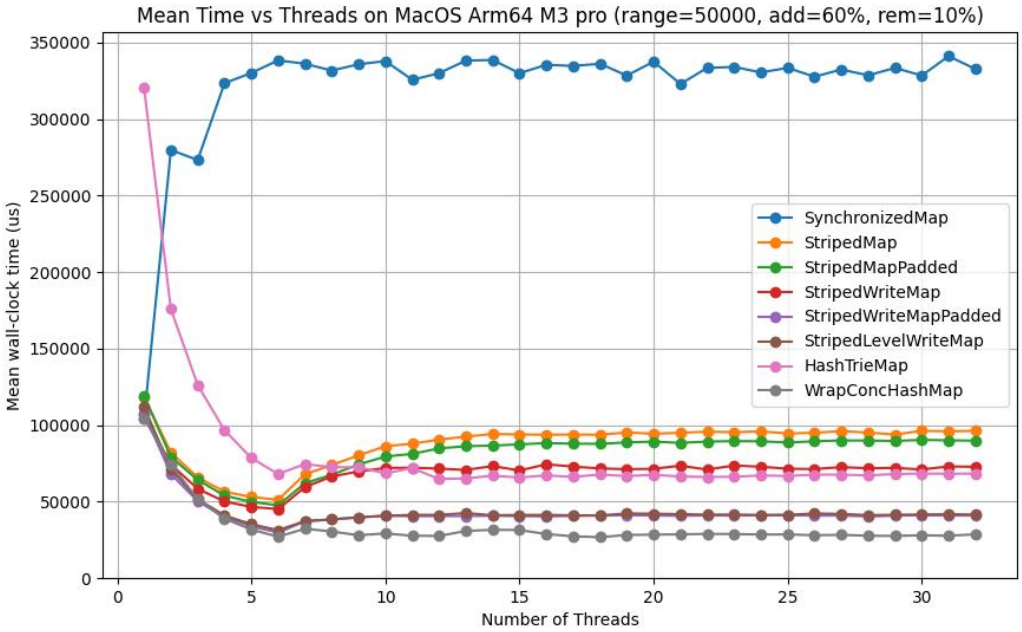


Figure 9.10: Time as function of number of workload threads, for all hash map implementations; lower is better; ; `put` 60 % and `remove` 10 %; Apple M3 pro.

- Figures 9.10 and 9.15 show that for a single thread `N=1`, almost all hash map implementations are equally fast, taking around 110 ms for 5 million executions of the for-loop body above, or 22 ns per iteration. Only HM7 HashTrieMap at 320 ms for 5 million executions is three times slower, presumably because it is based on a tree data structure which requires more memory accesses than arrays; on the other hand it scales quite well for large thread counts.

- The HM1 SynchronizedMap is not scalable at all; the more threads access it, the slower it becomes until it reaches a plateau at four threads.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

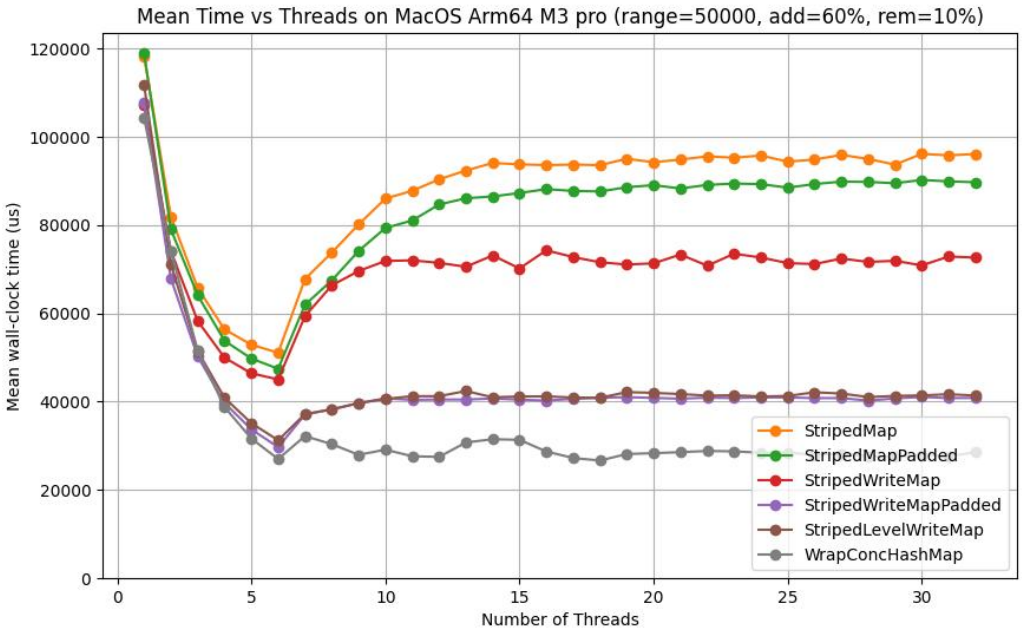


Figure 9.11: Time as function of number of workload threads, for hash map implementations except HM1 and HM7; lower is better; `put` 60 % and `remove` 10 %; Apple M3 pro.

- Disregarding HM1 and HM7 and focusing on the six other implementations in figure 9.11 we see that all become faster with more workload threads until `N=6`, which is the number of performance cores in the Apple M3pro CPU; see figure 8.7.

- For more than `N=6` workload threads, the implementations fall in two groups. Those in the first group — HM2 StripedMap, HM3 StripedMapPadded and HM4 StripedWriteMap — all become slower with more than `N=6` workload threads until they reach a plateau at `N=12` approximately. Those in the second group — HM5 StripedWriteMapPadded, HM6 StripedLevelWriteMap and HM8 WrapConcHashMap — become only very slightly slower with increasing number of workload threads, and HM8 WrapConcHashMap is almost completely unaffected, which shows it professional design. In general, those in the second group are 2–3 times faster than those in the first group. The latter group uses immutable data and padding to avoid false sharing and cache line invalidation.

- Overall, the measurements confirm that using a single central lock (HM1) does not scale at all; that using multiple locks (HM2 and HM4) benefits performance only in combination with padding or other ways to avoid false sharing (HM3 and HM5); that judicious use of immutable data combined with padding (HM5) or a naturally padded data structure (HM6) can achieve good performance; that using compare-and-set instead of locks does not guarantee performance but can give good scalability; and finally that Java's ConcurrentHashMap (HM8) is very professionally engineered.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

9.3.12 Hash map measurements, AMD EPYC 7313

This section analyses the performance of the eight hash map implementations on the server CPU AMD EPYC 7313 whose structure is shown in figure 8.8.

To a large extent the measurements confirm those from the laptop M3 pro CPU in section 9.3.11, but with several new aspects:

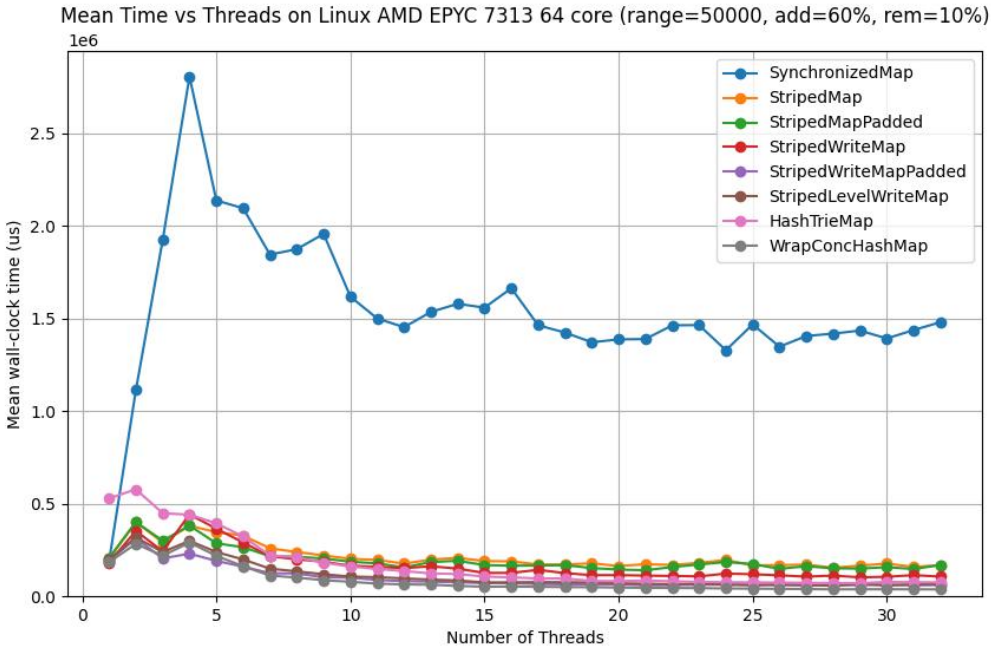


Figure 9.12: Time as function of number of workload threads, for all hash map implementations; lower is better; `put` 60 % and `remove` 10 %; AMD EPYC 7313.

- Figures 9.12 and 9.15 show that for a single thread `N=1`, almost all hash map implementations take around 200 ms for 5 million executions of the for-loop body above, or 40 ns per iteration. Only HM7 HashTrieMap at 528 ms is slower, but scales quite well for large thread counts.

- The HM1 SynchronizedMap is not scalable at all; the more threads access it, the slower it becomes until it reaches a plateau at four threads.

- Disregarding HM1 and HM7 and focusing on the six other implementations in figure 9.13 we find there is considerable fluctuations for small thread counts `N` from 2 to around 6. This is not necessarily measurement noise, but could reflect the complex and inhomogeneous structure of the CPU as shown in figure 8.8. Maybe the few active threads run on cores in different blocks or packages of the CPU and communication therefore is slow. For higher thread counts, more threads will necessarily run in the same block or package. This is unclear.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

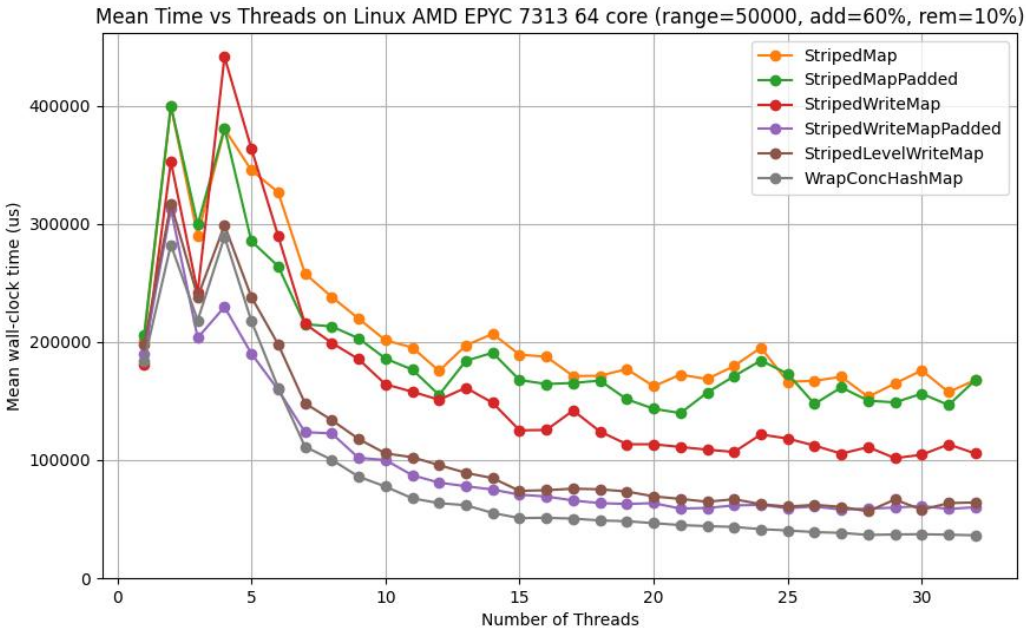


Figure 9.13: Time as function of number of workload threads, for hash map implementations except HM1 and HM7; lower is better; `put` 60 % and `remove` 10 %; AMD EPYC 7313.

- For thread counts higher than 5, we see that the six implementation all become faster with more workload threads, and as in the laptop CPU, the implementations fall in two groups. Those in the first group — HM2 StripedMap, HM3 StripedMapPadded and HM4 StripedWriteMap — show very little speed-up with higher thread count.

  Those in the second group — HM5 StripedWriteMapPadded, HM6 StripedLevelWriteMap and HM8 WrapConcHashMap — continue to become faster with increasing number of workload threads. Those in the second group are twice as fast as those in the first group, with HM8 ConcurrentHashMap almost twice as fast again as HM5 and HM6.

- Overall, the measurements confirm the observations from the laptop CPU, but with the surprising additional aspect that all implementations except HM7 HashTrieMap become slower, not faster, for low thread counts 2–5, even Java's ConcurrentHashMap.

- Perhaps it is simply not possible to efficiently support high update counts (probability of `add` being 60 % and of `remove` 10 %) on a shared-memory multi-core machine as complex as the AMD EPYC. This hypothesis is supported by the measurements shown in figure 9.14 where the `remove` probability has been lowered to 2 %, considerably reducing the number of memory updates performed. In that workload, the HM4, HM5, HM6 and HM8 all show much less slowdown for small thread counts, and better scalability, than when the `remove` probability was 10 %.

The concurrent hash map results from both the laptop and server CPUs are summarised in figure 9.15. It can be seen that both lock-based and CAS-based data structures can be scalable,

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

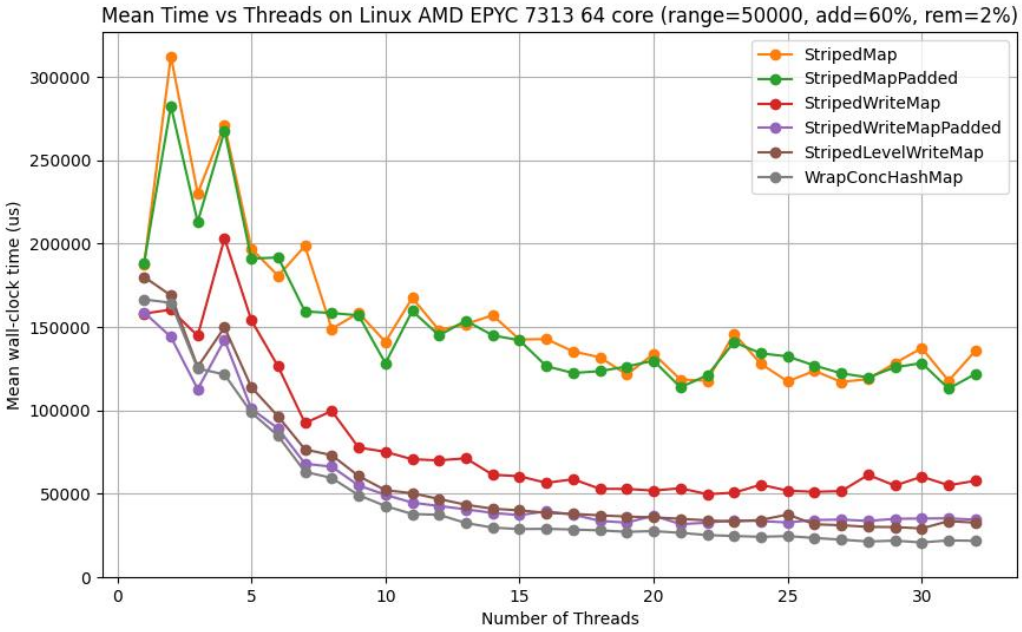


Figure 9.14: Time as function of number of workload threads, for all hash map implementations; lower is better; `put` 60 % and `remove` 2 %; AMD EPYC 7313.

			M3pro			AMD			
Name	Sync	Local	1	32	Spd	1	32	Spd	Scble
HM1 SynchronizedMap	Lock	No	104	333	0.31	187	1479	0.13	No
HM2 StripedMap	Lock	No	118	96	1.23	202	168	1.21	No
HM3 StripedMapPadded	Lock	Yes	119	90	1.33	206	168	1.23	No
HM4 StripedWriteMap	Lock	No	107	73	1.48	181	105	1.71	No
HM5 StripedWriteMapPadded	Lock	Yes	108	41	2.64	190	60	3.16	Yes
HM6 StripedLevelWriteMap	Lock	Yes	112	41	2.70	198	64	3.08	Yes
HM7 HashTrieMap	CAS	Yes	320	68	4.69	528	74	7.10	Yes
HM8 WrapConcHashMap	Lock/CAS	Yes	104	29	3.66	184	36	5.06	Yes

Figure 9.15: Hash map implementations and their synchronization mechanism, cache-locality of updates, performance (ms per 5 million operations for 1 and 32 threads), speedup factor from 1 to 32 threads, and assessed scalability; workload `put` 60 % and `remove` 10 %.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

but memory updates must be cache-local. The CAS-based HM7 HashTrieMap scales well but is slow in single-threaded execution.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

9.4 Parallel k-means clustering and cache coherence

This section investigates how an "obvious optimization" can make a concurrent Java program considerably slower. We demonstrate that the culprit is the hardware: false sharing of CPU cache lines and the attendant cache coherence protocol overhead, as hinted at in section 8.7.

The algorithm we consider is *k-means clustering*, which takes as input a set of `N` points in the plane and a desired number `k` of clusters, and outputs the best grouping of those points into `k` clusters. The algorithm maintains a preliminary clustering and repeatedly performs both an assignment step and an update step to create a new preliminary clustering. The *assignment step* puts each point in its nearest cluster; the *update step* recomputes the new mean, also called centroid, of each resulting cluster. This goes on until the assignment-update step does not change the preliminary clustering; then it is final.

In the example considered here, we have `N` = 200,000 points in the plane and `k` = 81 desired clusters, and its takes 108 assignment-update steps to reach the final clustering.

In this algorithm we cannot parallelise the 108 iterations because each depends on the preliminary clustering computed by the previous step. But we can parallelise the processing of the 200,000 points in each assignment step and each update step.

9.4.1 The mystery

First we show that some seemingly "improved" code may be considerably slower than the corresponding unimproved code. The unimproved parallel code, called KMeans2P, looks like this, with some details left out. The "`let taskCount parallel tasks do`" syntax is pseudocode for creating and starting `taskCount` parallel tasks, each processing a segment `[from...to-1]` of the `points` array:

```
while (!converged) {
  // Assignment step: put each point in exactly one cluster
  let taskCount parallel tasks do {
    final int from = ..., to = ...;
    for (int pi=from; pi<to; pi++)
      myCluster[pi] = closest(points[pi], clusters);
  }
  // Update step: recompute mean of each cluster
  let taskCount parallel tasks do {
    final int from = ..., to = ...;
    for (int pi=from; pi<to; pi++)
      myCluster[pi].addToMean(points[pi]);
  }
  converged = true;
  for (Cluster c : clusters)
    converged &= c.computeNewMean();
}
```

In the assignment step, `taskCount = 48` tasks collaborate to go through `N` = 200,000 points and for each such point `points[pi]` find the cluster whose mean is closest to the point, and store a reference to that cluster in `myCluster[pi]`. The call `closest(p, clusters)` returns the cluster whose mean is closest to point `p`; see code on page 150.

In the update step, the same number of tasks collaborate to traverse the `points[]` and `myCluster[]` arrays at the same time, calling `addToMeans` to add each point `points[pi]` to the

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

(previously computed) closest cluster's `sumx` and `sumy` and `count` fields (see below). Method `addToMeans` takes a lock on the cluster before the field updates.

Now it would seem simpler and faster to just do the `addToMean(p)` operation inside the assignment step, instead of letting the assignment step write to the `myCluster` array and letting the update step traverse the array later on. Then the update step's only work would be to compute the new means for the 81 clusters, and one can get rid of the entire `myCluster` array and the 200,000 writes to it.

The resulting shorter parallel code, called KMeans2Q, would look like this:

```
while (!converged) {
  // Assignment step: put each point in exactly one cluster
  let taskCount parallel tasks do {
    final int from = ..., to = ...;
    for (int pi=from; pi<to; pi++)
      closest(points[pi], clusters).addToMean(points[pi]);
  }
  // Update step: recompute mean of each cluster
  converged = true;
  for (Cluster c : clusters)
    converged &= c.computeNewMean();
}
```

Indeed the simpler KMeans2Q code is 5 percent faster than KMeans2P *when executed sequentially*, with `taskCount = 1`. Here's the average wall clock time of 10 runs, each processing 200,000 points into 81 clusters in 108 iterations of the `while` loop:

Class	Time (sec)
KMeans2P	4.240
KMeans2Q	4.019

However, when running multiple parallel tasks with `taskCount = 8`, the "optimized" KMeans2Q is 70 percent slower than the apparently wasteful KMeans2P which first writes the array and then reads it:

Class	Time (sec)
KMeans2P	1.310
KMeans2Q	2.234

This is for a 4-core (and hyperthreading, so 8 hardware threads) shared-memory CPU, an Intel i7-4870HQ running MacOS and Oracle's Java SE Hotspot 1.8.0.

On a large Windows 10 server with an Intel Xeon E5-2680 v3 that has 2 x 12 cores (and hyperthreading, so 48 hardware threads) at 2.5 GHz, the slowdown from KMeans2P to KMeans2Q is even more remarkable when `taskCount = 48`:

Class	Time (sec)
KMeans2P	0.853
KMeans2Q	6.587

On that server CPU, the "optimized" KMeans2Q program is a whopping 7.7 times slower than the initial KMeans2P. Slowdowns by a factor of 2 are observed also on the laptop and server CPUs presented in section 8.6.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

9.4.2 Explaining the slowdown

The reason for the slowdown is somewhat subtle: it is caused by the CPU's cache coherence protocol and false sharing of cache lines, as discussed in section 8.7.

The Point class is immutable, so presumed innocent:

```
class Point {
  public final double x, y;
  ...
}
```

The nested Cluster class has four mutable fields and some methods:

```
static class Cluster extends ClusterBase {
  private volatile Point mean;
  private double sumx, sumy;
  private int count;
  public synchronized void addToMean(Point p) {
    sumx += p.x;
    sumy += p.y;
    count++;
  }
  public synchronized boolean computeNewMean() { ... }
}
```

The `addToMean` method takes a lock for threadsafe update of the `sumx`, `sumy` and `count` fields. The `mean` field is updated only by the `computeNewMean` method, and is volatile for visibility of updates even without locking.

The method `closest(p, clusters)` traverses the `clusters` array and returns the cluster whose `mean` is closest to point `p`:

```
private static Cluster closest(Point p, Cluster[] clusters) {
  Cluster bestCluster = null;
  double bestDist = Double.POSITIVE_INFINITY, dist;
  for (Cluster c : clusters) {
    dist = p.sqrDist(c.mean);
    if (dist < bestDist) {
      bestCluster = c;
      bestDist = dist;
    }
  }
  return bestCluster;
}
```

The KMeans2P assignment step reads the `points` array, the `clusters` array, and the Point objects referred by the clusters' `mean` fields. It writes to the `myCluster` array, each assignment task writing only to its own `[from...to-1]` segment of that array. The assignment step takes no locks at all.

The KMeans2P update step reads the `points` array, the `myCluster` array, each cluster's `mean` field, and the `x` and `y` fields of the referred-to Point objects. It updates the 81 clusters' `sumx`, `sumy` and `count` fields after taking the cluster's lock. Each of the 81 locks is taken on average $200{,}000/81 = 2469$ times.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

The KMeans2Q assignment step also reads the `points` array and the `clusters` array and takes the 81 cluster locks the same number of times (and it avoids writing and reading the `myCluster` array, and in contrast to KMeans2P traverses the `points` array just once). Hence the slowdown cannot be attributed to additional lock taking, increased lock contention, increased memory usage or the like. In fact, memory usage decreases by 1.6 MB, the size of the `myCluster` array.

The problem in KMeans2Q is that the 200,000 memory updates to the 81 cluster objects' `sumx`, `sumy` and `count` fields are interleaved with the $81 \cdot 200{,}000 = 16.2$ million read accesses to the cluster objects' `mean` fields.

Namely, it seems plausible that the JVM runtime layouts a Cluster object's fields like this:

mean	sumx	sumy	count

or perhaps some other permutation of the four fields, but in any case the fields fit into a single 64-byte cache line. Whenever a CPU core wants to write the `sumx`, `sumy` or `count` field during a call to the `addToMean` method, that core will have to put the cache line into the E(xclusive) state and then the M(odified) state of the MESI cache coherence protocol, as described in section 8.7. This will cause the entire cache line to be invalidated in all other cores' caches, invalidating also their cached copy of the `mean` field — just because they are on the same cache line. Thus the cache line containing `mean` repeatedly switches between the E(xclusive), M(odified) and I(nvalid) states. This is unavoidable, and happens also in the KMeans2P update step, but there it is quite harmless because the `mean` fields are not accessed at all. During the KMeans2Q assignment step, however, the `mean` field of each cluster will be read 200,000 times (once for each point). The cache line invalidations caused by `addToMean` give rise to cache misses on subsequent read accesses which will therefore be much slower, by a factor of maybe 100, than an L1 cache hit.

To validate the hypothesis that false cache line sharing causes the slowdown, we can pad the Cluster object with dummy fields in the hope that they prevent the `mean` field from being on the same cache line as the `sumx`, `sumy` and `count` fields at runtime:

```
static class Cluster extends ClusterBase {
  private volatile Point mean;
  private long dummyy1, dummyy2, dummyy3, ..., dummyy16;
  private double sumx, sumy;
  private int count;
  private long dummy1, dummy2, dummy3, ..., dummy16;
  ...
}
```

The expectation is that the padding produces a Cluster memory layout like this, which ensures that the `mean` field is on a different cache line that `sumx`, `sumy` and `dummy`:

mean	dy1	...	dy16	sumx	sumy	count	d1	...	d16

And indeed the padding dramatically improves the KMeans2Q performance on both the i7 and the Xeon, actually making their performance on the i7 indistinguishable:

Class	i7 (sec)	Xeon (sec)
KMeans2P	1.344	1.055
KMeans2Q padded	1.342	1.469

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

So indeed it seems that false cache line sharing is involved in the slowdown. Note that the dummy fields work only because the JVM runtime does not optimize them away, which it might, and because it apparently does not reorder the fields aggressively. In Java, there is a field annotation @jdk.internal.vm.annotation.Contended that adds such cache line padding, but it is for internal use in the Java class libraries only.

The reason for having two groups of dummy fields is that we want to prevent one cluster's `mean` field from sharing a cache line with the previous cluster's `count` field, which could happen if the cluster objects are allocated next to each other in the heap:



9.4.3 Functional stream programming sidesteps the problem

There seems to be a simple way to avoid these subtleties altogether: Use Java functional parallel streams instead of side-effects, tasks and locks. Here is a k-means implementation, called KMeans3P, using Java streams and functional programming:

```
while (!converged) {
  { // Assignment step: put each point in exactly one cluster
    final Cluster[] clustersLocal = clusters; // For capture in lambda
    Map<Cluster, List<Point>> groups =
      Arrays.stream(points).parallel()
            .collect(Collectors.groupingBy(p -> closest(p, clustersLocal)));
    clusters = groups.entrySet().stream().parallel()
      .map(kv -> new Cluster(kv.getKey().getMean(), kv.getValue()))
      .toArray(Cluster[]::new);
  }
  { // Update step: recompute mean of each cluster
    Cluster[] newClusters =
      Arrays.stream(clusters).parallel().map(Cluster::computeMean)
            .toArray(Cluster[]::new);
    converged = Arrays.equals(clusters, newClusters);
    clusters = newClusters;
  }
}
```

now with completely immutable clusters and no side effects in methods:

```
static class Cluster extends ClusterBase {
  private final List<Point> points;
  private final Point mean;
  public Cluster(Point mean, List<Point> points) {
    this.mean = mean;
    this.points = Collections.unmodifiableList(points);
  }
  public Cluster computeMean() {
    double sumx = points.stream().mapToDouble(p -> p.x).sum(),
           sumy = points.stream().mapToDouble(p -> p.y).sum();
    Point newMean = new Point(sumx/points.size(), sumy/points.size());
    return new Cluster(newMean, points);
  }
}
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Interestingly, the functional parallel stream version KMeansP3 performs extremely well compared to KMeans2P and the unpadded KMeans2Q, and this is the case both for the 4-core i7, the 24-core Xeon and the 12- and 32-core machines described in section 8.6 (times in seconds):

Class	i7	Xeon	M3 pro	AMD 7313
KMeans2P, original parallel	1.310	0.853	0.664	0.910
KMeans2Q, "optimized"	2.234	6.587	1.119	1.944
KMeans3P, functional streams	1.264	0.553	0.339	0.257

There are at least three reasons for the excellent parallel performance of this functional stream-based solution. First, the Java stream library and its parallelization are very well engineered and probably play well together with the JVM just-in-time compiler. Second, the absence of side effects in the user code obviously means that it does not cause cache line invalidation, since all shared data are immutable and so the cache lines can remain in the S(hared) state of the MESI cache protocol, without any invalidation messages. Third, the JVM runtime system's memory allocator and garbage collector obviously must allocate and initialize objects, and reclaim the memory when the objects die, which requires writing to memory. But apparently allocation and collection have been engineered to do that without incurring much cache coherence overhead, while still permitting a high degree of parallelism.

9.4.4 Observations

A code change that is an "obvious improvement" in a sequential setting can considerably slow down the code when executed in parallel on a multicore architecture.

The reason for the slowdown were the cache coherence protocol and especially false cache line sharing, as demonstrated by removing the false sharing and restoring good performance.

Reimplementing the same algorithm using Java functional stream programming and immutable data structures achieves the best parallel performance on all the four tested hardware platforms.

This points to a way out of the performance complexity of shared-memory multi-core computers: for parallel efficiency, use only immutable data and functional stream programming where possible.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15